AAtt-CNN: Automatic Attention-Based Convolutional Neural Networks for Hyperspectral Image Classification

Mercedes E. Paoletti, *Senior Member, IEEE*, Sergio Moreno-Álvarez, *Graduate Student Member, IEEE*, Yu Xue, *Member, IEEE*, Juan M. Haut, *Senior Member, IEEE*, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Convolutional models have provided outstanding performance in the analysis of hyperspectral images (HSIs). These architectures are carefully designed to extract intricate information from nonlinear features for classification tasks. Notwithstanding their results, model architectures are manually engineered and further optimized for generalized feature extraction. In general terms, deep architectures are time-consuming for complex scenarios, since they require fine-tuning. Neural architecture search (NAS) has emerged as a suitable approach to tackle this shortcoming. In parallel, modern attention-based methods have boosted the recognition of sophisticated features. The search for optimal neural architectures combined with attention procedures motivates the development of this work. This article develops a new method to automatically design and optimize convolutional neural networks (CNNs) for HSI classification using channel-based attention mechanisms. Specifically, 1-D and spectral–spatial (3-D) classifiers are considered to handle the large amount of information contained in HSIs from different perspectives. Furthermore, the proposed automatic attention-based CNN (`AAtt-CNN`) method meets the requirement to lower the large computational overheads associated with architectural search. It is compared with current state-of-the-art (SOTA) classifiers. Our experiments, conducted using a wide range of HSI images, demonstrate that `AAtt-CNN` succeeds in finding optimal architectures for classification, leading to SOTA results.**

***Index Terms*— Convolutional neural networks (CNNs), deep learning (DL), hyperspectral images (HSIs), neural architecture search (NAS).**

I. Introduction

HYPERSPECTRAL images (HSIs) comprise data with a large amount of pixelwise spectral information spatially arranged. Spectral features are concentrated in a large number (hundreds) of strongly correlated spectral bands, where each one covers a fixed range of wavelengths. This spectral information is captured throughout the electromagnetic spectrum and extended beyond the range of visible colors. In fact, the materials found on the ground are represented in each pixel by absorbing the solar radiation reflected by the terrestrial surface and generating a spectral signature that is represented by a C-dimensional vector, where C is the number of bands. With the goal of exploiting the rich information contained in every single pixel, the process of extracting material features has become a major subject of study for domain-relevant fields, such as precision agriculture [1], [2], crop management [3], environmental urban planning [4], soil estimation [5], hydrological analysis [6], and other environmental applications [7]. Furthermore, higher spectral, spatial, and temporal resolutions enhance the ability to accurately determine objects in the gathered HSI data cubes. Recently, improvements to both sensing instruments and processing tools have been conducted to improve the visualization and representation of materials and objects in the scene, including techniques, such as super-resolution [8], [9] or noise reduction [10], among others [11]. Also, these techniques address some of the drawbacks related to the inherent complexity introduced by the high spatial and spectral resolution of HSIs. Of these methods, classification is a hot topic in HSI data processing. In particular, recent advances in deep learning (DL) have provided unparalleled results. Notwithstanding their outstanding accuracy, the selection of a suitable deep architecture is not without some challenges, as it is a purely handcrafted process in which the knowledge of the designer weighs heavily. This article focuses on providing an automatic solution to the design and configuration of deep networks for HSI analysis, through the improvements of the neural architecture search (NAS) procedure using current state-of-the-art (SOTA) methods, such as attention-based strategies.

A. HSI Classification

Image classification is a process in which feature extraction is first performed, and a subsequent classification stage (that exploits the obtained features to determine the class membership for each pixel) is conducted. In HSI classification, both spectral information and spatial information are used for feature extraction and classification purposes. Traditionally,

Manuscript received 2 February 2023; revised 6 March 2023; accepted 13 March 2023. Date of publication 3 May 2023; date of current version 19 May 2023. This work was supported in part by the European Regional Development Fund "A Way to Achieve Europe" (FEDER) through the Junta de Extremadura under Grant IB20040; in part by the Consejería de Economía, Ciencia y Agencia Digital (Junta de Extremadura); in part by the Miniterio de Ciencia e Innovación under Grant PID2019-110315RB-I00/AEI/10.13039/501100011033; and in part by the Junta de Extremadura FEDER under Grant GR21040 and Grant GR21099. *(Corresponding author: Juan M. Haut.)*

Mercedes E. Paoletti, Juan M. Haut, and Antonio Plaza are with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, Escuela Politécnica, University of Extremadura, 10003 Cáceres, Spain (e-mail: juanmariohaut@unex.es).

Sergio Moreno-Álvarez is with the Department of Computer Systems Engineering and Telematics, Escuela Politécnica, University of Extremadura, 10003 Cáceres, Spain.

Yu Xue is with the School of Computer Science, Nanjing University of Information Science and Technology, Nanjing 210044, China.

Digital Object Identifier 10.1109/TGRS.2023.3272639

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

HSI classification algorithms focus on the information contained along the spectral dimension to define the boundaries between land-cover categories. Nonetheless, those methods that introduce spatial information have gained attention in recent years, outperforming pixelwise accuracy by smoothing the inaccuracies of spectral-based classifiers using neighboring pixel information, which reduces the spectral variability and uncertainty introduced by both few labeled training samples and a skewed distribution of class labels [12], [13]. Nevertheless, samples selection has been a subject of study with the aim of selecting the most appropriate ones to enhance the available training collection. Indeed, these techniques address the issue of a limited number of training samples [14], [15]. Also, band sampling has been widely studied to reduce the spectral training space [16].

1) Traditional HSI Classifiers: Classification approaches are mainly categorized into supervised and unsupervised algorithms. The former ones train their parameters based on the information provided by pairs of training pixel labels that define the materials in the scene. Consequently, training is guided by labels to produce more accurate predictions. On the contrary, unsupervised methods do not require labels during the training, since they capture both geometry- and distance-based patterns through encoded process [17]. In both cases, the objective is to perform successful feature discrimination to improve the learning procedure. Examples of successful classification techniques include support vector machines (SVMs) [18], logistic regression (LR) [19], sparse autoencoder (SAE) [20], which implement single-layer classifiers that can successfully learn data properties, graph-based networks, such as graph convolutional networks (GCNs) [21], and multilevel superpixel structured graphs U-Net (MSSGU) [22].

2) DL Classifiers: Recently, DL architectures have demonstrated outstanding performance in the task of extracting abstract and complex features from HSIs by using multilayer neural networks that significantly enhance feature extraction and exploitation [23], [24], [25], [26]. Specially, convolutional neural networks (CNNs) apply a moving kernel as a receptive field throughout the input image, leading to accurate classification results [27], [28]. The procedure involves a series of linear transformations and introduces nonlinear functions over the hierarchy of computational layers. Other techniques, such as morphological transformations (MTs) [29] or principal component analysis (PCA) [30], [31], have been combined with the aforementioned approaches to boost the classification results. In this regard, MT has been used to identify relevant features over the spatial domain and successfully applied to several areas of computer vision [32]. Meanwhile, PCA provides accurate noise reduction by selecting the most representative components of the image after a transformation process.

3) Strengths of CNN Classifiers: The major strength of CNN-based approaches lies in the local relationships; i.e., all inputs fed to a given neuron are close to each other, and the weights are shared across the receptive field of neurons (filters). Regarding the classification of HSIs, convolutional models offer a wide range of variations in terms of implementations, i.e., regarding the convolutional direction and the output shape. Specifically, 1-D CNNs and

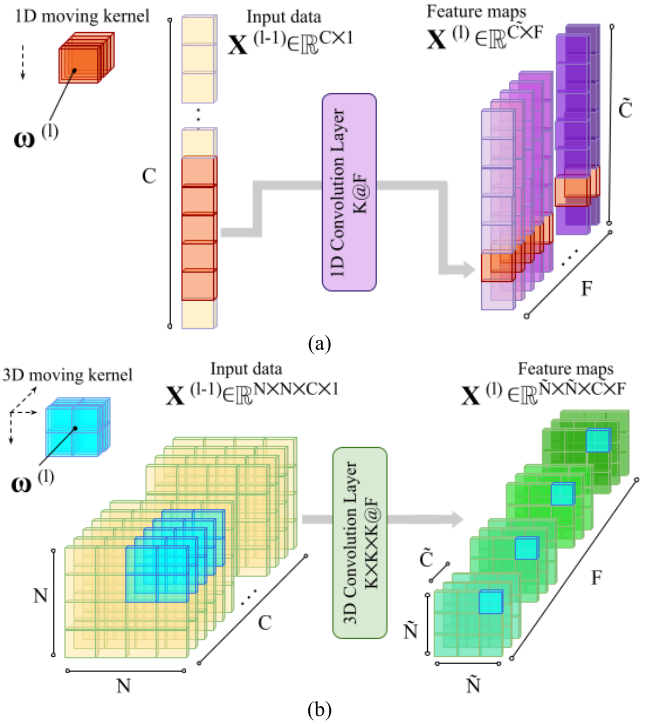


Fig. 1. Sliding a convolutional kernel though the input HSI $\mathbf{X}^{(l-1)}$. The output feature maps $\mathbf{X}^{(l)}$ are generated in layer (l) to be given as input to $(l+1)$. Note the different shapes of the input data and feature maps for both 1-D and 3-D convolutions. Kernel weight values $\boldsymbol{\omega}$ are represented through the kernel application space $K@F$ and $K \times K \times K@F$, where k and f denote the window size and the number of filters, respectively. Kernel applications are highlighted for each filter in red and blue for (a) 1-D and (b) 3-D convolution layers, respectively.

spectral–spatial (3-D) CNN approaches, shown in Fig. 1, have recently gained importance based on their high performance offered. The 1-D CNNs apply layer kernels over the spectral dimension (i.e., in pixelwise fashion). In contrast, 3-D CNNs slide a kernel along both the spatial and spectral dimensions, producing a variable feature map at the layer outputs. Note that the input data in HSIs are 3-D in nature, and therefore, 3-D CNNs are ideally designed to extract spectral–spatial features [33], [34]. These architectures have been used in a significant number of works within the current literature. These spectral and spatial features are combined to achieve better HSI classification results [35]. For instance, Roy et al. [36] propose a combination of 3-D and 2-D convolutions to enhance the feature representation. Besides, techniques, such as generative adversarial minority oversampling (GAMO) [37]—for data augmenting from class-specific samples—or GCNs [38]—which model relationships among data samples by using a graph structure—have been recently demonstrated to provide notable results for HSI classification purposes. In addition, spectral–spatial fusion strategies have been shown to offer advantages for HSI classification over spectral-only and spatial-only classifiers [39].

4) Spectral Versus Spatial–Spectral CNNs: The 1-D CNNs, i.e., spectral classifiers, perform the classification of each pixel vector independently. These spectral classifiers aim to accurately discriminate captured materials by extracting high-level abstract spectral features with a simple and straightforward implementation. For example, Slavkovikj et al. [40] use a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

mixture of spatial and spectral information through a neighborhood window with a kernel size of 1. On the other hand, 3-D CNN models have improved classification remarkably due to their strong ability to extract spectral–spatial features. In fact, 3-D CNNs provide a robust and reliable feature extraction mechanism due to their ability to handle both spatial and spectral information simultaneously. This has been recently demonstrated by Lin et al. [41] in comparison with other 2-D CNN methods. This feature extraction process is shown in (1) for the calculation of the output feature map $\mathbf{X}^{(l)}$ given the lth operational layer of a 3-D CNN, as illustrated in Fig. 1(b)

$$\mathbf{X}^{(l)} = \left(\boldsymbol{\omega}^{(l)}, \mathbf{X}^{(l-1)}, \mathbf{b}^{(l)}\right)_{K,K,K,F}. \tag{1}$$

While the 1-D and 2-D convolutions define the 1-D ($\boldsymbol{\omega} \in \mathbb{R}^{K@F}$) and 2-D convolution kernels ($\boldsymbol{\omega} \in \mathbb{R}^{K\times K@F}$), respectively,[1] the 3-D convolution specifies a tridimensional kernel $\boldsymbol{\omega} \in \mathbb{R}^{K\times K\times K@F}$, defining F filters with a spatial–spectral convolution window of size $K \times K \times K$ (the height, width, and depth of the window, respectively). Consequently, the number of convolution directions directly affects the shape of the data. Thus, considering hyperspectral data cubes, the 1-D convolution receives batches of vectors with C channels, taking into account only the spectral domain, so the spatial relationships between pixels are lost; the 2-D convolution receives batches of $N \times N$ pixels with C channels, where the depth of the layer is equal to the depth of the input data, i.e., C, thereby reducing the expressiveness of the model by treating the entire spectrum as a whole, and finally, the 3-D convolution processes batches of $N \times N \times T \times C$ data, where $N \times N$ denotes the height and width of the scene, and C and T defines the channels and the "frames" or "time steps" of the data. Usually, to operate correctly on HSI, these dimensions are rearranged as $N \times N \times C \times 1$ to apply the filters to the spectral dimension. In this context, let us denote $\mathbf{X}^{(l-1)} \in \mathbb{R}^{N\times N\times C\times 1}$ as the input HSI data for the lth 3-D convolution layer.[2] Its convolution weights are denoted as $\boldsymbol{\omega}^{(l)} \in \mathbb{R}^{K\times K\times K@F}$, and bias is denoted as $\mathbf{b}^{(l)}$. The receptive field of the lth layer is determined by the kernel application $K \times K \times K$ over the input data $\mathbf{X}^{(l-1)}$ for the F filters following (1). This operation generates the output feature maps $\mathbf{X}^{(l)} \in \mathbb{R}^{\tilde{N}\times\tilde{N}\times\tilde{C}\times F}$, where spatial and spectral dimensions $\tilde{N} \times \tilde{N} \times \tilde{C}$ depends on several hyperparameters (such as the kernel size, the stride, and the padding). In particular, the value of a specific output coordinate (x, y, z) produced by any filter in a 3-D convolution can be defined as $x^{(l)}_{(x,y,z)}$, $\forall x \in [0, \tilde{N})$, $\forall y \in [0, \tilde{N})$, and $\forall z \in [0, \tilde{C})$, from

$$x^{(l)}_{(x,y,z)} = b^{(l)}_{(x,y,z)} + \sum_{i\in\mathcal{K}}\sum_{j\in\mathcal{K}}\sum_{t\in\mathcal{K}} x^{(l-1)}_{(x+i,y+j,z+t)}\omega^{(l)}_{i,j,t} \tag{2}$$

where $\mathcal{K} = [-K/2, K/2]$ is the mesh built from the size of the convolution kernel K.

[1]Note that the 1-D convolution defines F filters, i.e., the number of output filters in the convolution, of width K, which specifies the length of the spectral convolution window. Similarly, the 2-D convolution defines F filters with size $K \times K$, specifying the height and width of the spatial convolution window. In this sense, when implementing a convolutional kernel, a dimension (i.e., the number of filters F) is always added to it.

[2]Note that the input to the first layer $l = 0$ has dimensions $N \times N \times C \times 1$, where C is the original spectral dimension, as it is pixel-centered cropped patches of the original scene $H \times W \times C$.

The process is repeated layer by layer, extracting deeper properties as the model progresses. Hence, discovering complex abstract features involves the development of deep models. As a consequence, several problems are derived from the usage of deep CNNs for HSI processing.

5) Drawbacks of CNN Classifiers: A typically encountered problem is the limited availability of labeled data. This results in an important lack of training samples and, thus, in an inefficient training, leading to inadequate prediction and overfitting issues [42]. This shortcoming is usually tackled by data normalization techniques [43], regularization [44], [45], or data augmenting [46], among others.

Moreover, the large amount of information contained in the spectral dimension introduces a severe challenge due to the large computational requirements, increasing the complexity of the classifier. This is even compounded by the large intraclass variability of spectral mixtures. As a solution, methods to reduce the uncertainties and noise of the spectral mixture are typically proposed [47].

Regarding the computational load, training operations are computationally demanding, since a high number of layers increase the depth of the model and, thus, the number of operations. Reducing the computational burden has recently been the subject of some studies [25], [48], [49].

Finally, data complexity has usually been managed by applying stride and padding between layers with pooling operations. However, these techniques do not provide knowledge about the distribution and characteristics of the data, nor do they analyze the correlation and redundancy of the data, as they are tools for summarizing spatial–spectral information. To cope with these limitations, there are alternatives, such as attention-based methods [50], [51], [52], which have been able to take advantage of this complexity.

Despite these advances, both the design of an architecture and the use of either technique to improve model accuracy/performance are strictly at the designer discretion, where the architecture implementation is a normally manual process, completely handmade, and relying on the designer knowledge of the context, the data, and the problem to be solved. This precludes the analysis of the suitability of the deep architectures to the HSI classification problem.

6) NAS as Feasible Solution: Using deep models to optimize feature extraction implies a careful network design. Thus, in order to develop suitable solutions and avoid the generic use of handmade models that might be not optimal, model generation through NAS has been proposed considering multiple search strategies [53], [54]. The goal of these algorithms is to automatically select the architecture that gives the most promising operations (see Fig. 2).

As a first approximation, evolutionary/genetic strategies heuristically discover the optimal solution by generating individuals in a population, each of which represents a specific architecture belonging to the set of possibilities established by the population, and iteratively mutating them, evaluating and selecting the most promising architectures, and discarding the worst ones [55], [56], [57], [58], [59]. Notwithstanding their results, evolutionary operators need to be carefully designed for the specific problems at hand. Reinforcement learning (RL)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

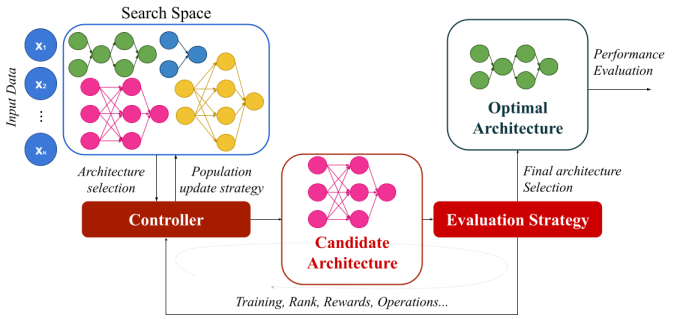


Fig. 2. Graphical overview of the NAS procedure from a discrete point of view. Starting from an initial search space, the procedure iteratively selects and evaluates possible architectures based on input data and constraints (evaluation strategy) and applies a series of operations/functions to update the search space. Finally, the architecture that best fits the requirements is selected.

models [60], [61] add to the heuristics the reward-penalty principle to determine the optimal architecture, providing the test stage accuracy as the feedback reward value [62], [63], [64]. This simplifies operation design at the expense of increased computational burden overall. In addition, alternative surrogate-based optimization approaches (such as Bayesian optimization strategies, sequential decision processes, random search, or Monte Carlo tree search) have also been studied in the literature [65], [66], [67], [68], [69], [70], [71]. From all these methods, the most popular strategy, and the focus of this work is the gradient-based technique. The procedure aims to optimize the hyperparameters of the architecture while learning connectivity patterns. As a consequence, the number of possible architecture configurations is very large, and the search for optimized architectures entails significant computational requirements [72]. To reduce the trade-off between computational cost and efficiency, it is sufficient to reduce the computational workload by providing an operations space that includes the most promising operations.

B. Related Work

NAS has become one of the most popular areas of study under the new automatic machine learning (AutoML) stream [73], [74], [75], with the goal of automating much of the decisions and tasks of designing and implementing machine learning pipelines that would traditionally be conducted by teams with specific expertise. It is not only remarkable its potential to handle a large space of network architectures under a mix of constraints and training data, but also its flexibility to produce a wide range of algorithm design variants [76]. Image classification approaches that use NAS-generated models have outperformed handmade architectures [77]. Indeed, NAS can successfully tackle the automatic design of complex architectures [77], [78], such as DenseNet [79] or ResNet [80]. Such approaches implement skip-connections operations, introducing identity and other functions of previous inputs to deeper layers. Moreover, these architectures ensure feature reusability while avoiding degradation in very deep networks.

RL is one of the most widely used strategies for generating and searching neural architectures. Specifically, Zoph and Le [63] focused on the automatic design of recurrent neural networks (RNNs). The resulting RNN is trained using RL techniques based on a reward (typically, the accuracy obtained by the architecture), feeding the training to produce better architectures. Alternatively, efficient NAS (ENAS) implements a high-performance automatic architecture design. In this method, an optimal subgraph, which defines the network operations, is selected by parameter-sharing techniques. The search space can be represented using a single directed acyclic graph (DAG), conducting parameter sharing through the information that flows between local node computations and edge connections. Recently, a progressive NAS method to learn the architecture of a CNN was designed by Liu et al. [70]. This work substantially improves the existing SOTA on RL and evolutionary algorithms. The method employs sequential model-based ensemble optimization (SMBO) based on the search for increasingly complex architectures using an auxiliary model to reduce the search space. Finally, CNN model generation for HSI classification was addressed by Chen et al. [81] in 1-D and 3-D search spaces. This work implements a gradient-based method to find the best performing architecture by using validation data as a reference to assess the performance and accuracy obtained. Furthermore, other examples of gradient methods have been also implemented in the literature [82], [83], [84].

Although these NAS approaches frequently produce outstanding results, the computing cost remains a tough issue to solve. Several solutions based on the differentiable architectural search technique (DARTS) have been proposed in the literature. DARTS reduces the computational cost by optimizing model parameters and the time required to completely traverse the search space. Each candidate network is determined by the most promising operations using the learned weights. Nevertheless, DARTS faces the problem of massive selection of skip-connection operations, as they are unweighted operations, which causes their selection to be unfair and collapses the performance of the overall architecture. In order to address this problem, several alternatives have been proposed. For instance, fair differentiable architecture search (FDARTS) [85] implements a collaborative operation competition to avoid unfair advantages between different types of operations. Partial channel differentiable architecture search (PC-DARTS) [86] presents partial channel connections to avoid memory overhead, decreasing the computational cost and improving stability in NAS. This is accomplished by sampling some selected parts of the model to explore the search space in a nonredundant way. Conducting a progressive scan of the architecture search space, progressive DARTS (PDARTS) [87] introduces an optimization process to address overfitting while avoiding accuracy drops. In this context, PDARTS adopts a search-space approximation and regularization, which relies on introducing dropout operations to cut skip connections. Finally, attention-based methods have been also adopted to improve the efficiency of DARTS in CNN operations. Specifically, Xue and Qin [88] presented a partial attentional DARTS (ADARTS) that selects a few image channels that best characterize the input data.

C. Contributions of Our Work

Given the aforementioned challenges, we propose a new channel-based attention mechanism for automatic NAS, with

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

the goal of developing efficient and effective complex convolution-based models for spectral–spatial HSI classification. The proposed method automatic attention-based CNN (AAtt-CNN) has the following contributions:

1) automatic and flexible CNNs generation for HSI data classification by using small amounts of training samples;
2) enhancement of the generated architecture, by alleviating instability in the search for operations and reducing degradation caused by unfairly selected (unweighted) operations;
3) integration of channel-based attention mechanism to extract the most promising features in the spectral domain;
4) complexity reduction by decreasing the number of trainable parameters in the generated architecture.

The rest of this article is organized as follows. Section II describes some basic concepts of NAS and generation procedures. Section III presents the proposed methodology based on NAS for spectral–spatial HSI classification. Section IV illustrates the performance of our newly developed AAtt-CNN in a wide rage of datasets and also provides comparisons with other SOTA methods. Finally, Section V concludes this article with some remarks.

II. Automatic CNN Architecture Design

DARTS processes the search space continuously to cope with the high computational requirements. It builds candidate architectures using three main aspects: 1) the *search space*, which comprises the architectures that can be represented using a predefined set of operations $\mathcal{O}$; 2) the *search strategy*, which determines the decision making for architecture selection; and 3) the *performance estimation*, which determines the suitability of a candidate architecture using unseen data. These components are combined to ensure a trade-off between search cost and performance.

DARTS establishes a global network of stacked cell units to form the desired convolution-based model, each one obtaining and propagating their outputs (feature maps) to the next cells. Regarding this, the cth cell is a subnetwork modeled as a DAG, which comprises M nodes, i.e., $[n_1, \ldots, n_M]$, each one encoding a latent representation resulting from the application of some operation, and connected through directed edges (i, j), $\forall i, j \in [1, M]$, $i \neq j$, each of which represents one of the possible transform operations contained in $\mathcal{O}$, i.e., the edge (i, j) points to the candidate operation to be applied on n_i. Thus, for each cell, the search procedure aims to select an operation from $\mathcal{O}$ to connect each pair of nodes, creating the data path that links the node features with the desired operations, and resulting in the final network architecture.

Taking into account that the inputs of the cth cell are the outputs of predecessor cells ($c-1$, $c-2$, and so on), all its nodes are obtained by a weighted combination of the operations connecting it to all its predecessor nodes, as indicated by (3), with the cell output obtained by some reduction operation, such as concatenation $[n_1 \frown n_2 \frown \ldots \frown n_M]$

$$n_j = \sum_{i<j} f_{i,j}(n_i). \tag{3}$$

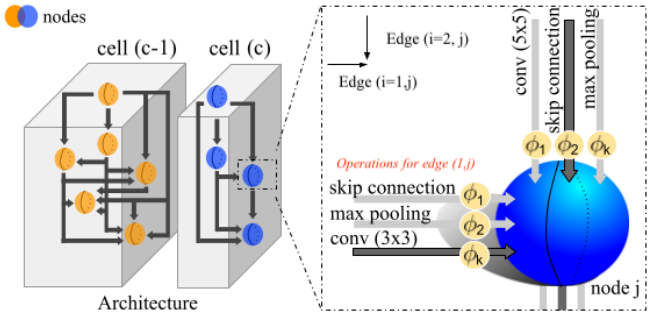


Fig. 3. NAS search space of cells $c-1$ and c (left). Detail of node n_j, connected to nodes 1 and 2 through edges $(1, j)$ and $(2, j)$. For each one, the potential operations to be allocated are represented with their corresponding weights (probabilities) $\phi^o_{i,j}$, $\forall o \in \mathcal{O}$, where the highest weight is highlighted in dark gray (right).

In this context, $f_{i,j}(\cdot)$ determines the operation to be assigned to the edge (i, j) from among the $|\mathcal{O}|$ operations available in $\mathcal{O} = [o_1, \ldots, o_{|\mathcal{O}|}]$ (i.e., convolutions, pooling, skip connections, and so on). This is illustrated in Fig. 3. For each edge, this function calculates the probability of assigning an operation $o \in \mathcal{O}$ to (i, j) over the L different possible operations by means of a softmax function, as indicated by the following equation:

$$f_{i,j}(n_i) = \sum_{o\in\mathcal{O}} \frac{\exp\left(\phi^o_{i,j}\right)}{\sum_{\tilde{o}\in\mathcal{O}} \exp\left(\phi^{\tilde{o}}_{i,j}\right)} o(n_i) \tag{4}$$

where learnable parameter $\phi^o_{i,j}$ weights the operation $o(n_i)$ on the edge connecting the ith and jth nodes. Consequently, for each edge (i, j), the vector of $|\mathcal{O}|$ operations weights $\boldsymbol{\phi}_{i,j}$ is obtained. It must be noted that the entire set of operation weights $\boldsymbol{\phi}$ encodes the underlying network architecture. As a result, the discrete architecture of each cell can be extracted by replacing each $f_{i,j}$ with the operation with highest probability given, i.e., $o_{i,j} = \text{argmax}_{o\in\mathcal{O}}\, \phi^o_{i,j}$.

The set of architecture parameters $\boldsymbol{\phi}$ and the network weights $\boldsymbol{\omega}$ are jointly used in the optimization step. In this regard, a bilevel optimization is performed in a nested manner, where the validation loss $\mathcal{L}_{\text{val}}$ (calculated on unseen data) is used to optimize $\boldsymbol{\phi}$, and the training loss $\mathcal{L}_{\text{tr}}$ (obtained on seen data) is used to optimize $\boldsymbol{\omega}$. The following equations provides the optimization problem:

$$\min_{\boldsymbol{\phi}} \mathcal{L}_{\text{val}}\left(\boldsymbol{\omega}^*(\boldsymbol{\phi}), \boldsymbol{\phi}\right) \tag{5a}$$

$$\text{s.t. } \boldsymbol{\omega}^*(\boldsymbol{\phi}) = \underset{\boldsymbol{\omega}}{\text{argmin}}\, \mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi}) \tag{5b}$$

where the values of $\boldsymbol{\omega}^*$ are the optimal network weights given the set of operation weights $\boldsymbol{\phi}$.

Note that this computation is applied through the entire set of cells and considering the whole space of operations. This entails a high computational burden and memory inefficiency, despite the efforts to be more efficient.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

A. Preliminaries: Optimization and Approximation

Although DARTS avoids discrete evaluation of each potential architecture by processing the search space in a continuous fashion, the high computational cost is still a challenging limitation. Each training iteration obtains the gradients $\nabla_{\boldsymbol{\omega}}$ considering the actual set of weights together with the wide range of possible operation combinations. This is time-consuming, with a complexity of $O(|\boldsymbol{\phi}||\boldsymbol{\omega}|)$, in addition to requiring a significant amount of memory to store the operations and their corresponding outputs. To deal with this issue, DARTS proposes an one-step training process that delays the complete optimization until the convergence of the architecture. Therefore, DARTS avoids the internal and costly optimization loop of (5b) at each training iteration. In particular, instead of obtaining the optimal weight $\boldsymbol{\omega}^*$ for each $\boldsymbol{\phi}$, i.e., $\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{val}}(\boldsymbol{\omega}^*(\boldsymbol{\phi}), \boldsymbol{\phi})$ (which involves two optimization loops), $\boldsymbol{\omega}^*(\boldsymbol{\phi})$ is approximated in a single SGD step as follows:

$$\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{val}}(\boldsymbol{\omega} - \xi\nabla_{\boldsymbol{\omega}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi}), \boldsymbol{\phi}) \tag{6}$$

where ξ is the learning rate for weight update. Consequently, in each epoch, parameters are optimized as follows:

$$\boldsymbol{\omega}' = \boldsymbol{\omega} - \xi\nabla_{\boldsymbol{\omega}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi}) \tag{7a}$$

$$\begin{aligned}
\text{1st order } (\xi = 0)\text{:}\ \boldsymbol{\phi} &= \boldsymbol{\phi} - \varepsilon\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}', \boldsymbol{\phi}) \\
\text{2nd order } (\xi > 0)\text{:}\ \boldsymbol{\phi} &= \boldsymbol{\phi} - \varepsilon\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{val}}(\boldsymbol{\omega} - \xi\nabla_{\boldsymbol{\omega}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi}), \boldsymbol{\phi}) \\
&= \boldsymbol{\phi} - \varepsilon\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{val}}(\boldsymbol{\omega}', \boldsymbol{\phi}) \\
&\quad - \xi \underbrace{\nabla^2_{\boldsymbol{\phi},\boldsymbol{\omega}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi})}_{\text{Hessian } \mathcal{H}} \nabla_{\boldsymbol{\omega}'}\mathcal{L}_{\text{val}}(\boldsymbol{\omega}', \boldsymbol{\phi})
\end{aligned} \tag{7b}$$

where ε is the learning rate for architecture weights. The Hessian product can be obtained through finite-difference estimation as indicated by (8), considering $\boldsymbol{\omega}^{\pm} = \boldsymbol{\omega} \pm \epsilon\nabla_{\boldsymbol{\omega}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}, \boldsymbol{\phi})$ and ϵ a small scalar [89]

$$\mathcal{H}\nabla_{\boldsymbol{\omega}'}\mathcal{L}_{\text{val}}(\boldsymbol{\omega}', \boldsymbol{\phi}) \approx \frac{\nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{tr}}(\boldsymbol{\omega}^+, \boldsymbol{\phi}) - \nabla_{\boldsymbol{\phi}}\mathcal{L}_{\text{tr}}(\boldsymbol{w}^-, \boldsymbol{\phi})}{2\epsilon}. \tag{8}$$

Search-space approximation and regularization aim to reduce the operation space using a score selection from previous stages. First, approximation determines the scores of the candidate operations while learning the architecture parameters $\boldsymbol{\phi}$, avoiding the optimization until convergence. Second, regularization aims to control the selection of skip connections. Indeed, skip connections usually lead to fast convergence of gradient descent optimization, since they are weight-free operations. For this purpose, the cooperative mechanism in (9) adopts a sigmoid $\sigma = 1/(1 + e^{-x})$ function to switch on/off operations without completely removing them

$$\hat{f}_{(i,j)}(n_i) = \sum_{o\in\mathcal{O}} \sigma\left(\phi^o_{i,j}\right) o(n_i). \tag{9}$$

As a consequence, a multihot encoding is implemented in order to promote the selection of promising operation and avoid the interference from unselected operations. The selected operations are ultimately included in the operation space.

B. Channel Attention Mechanism

Attention methods aim to improve the computational efficiency of DARTS, which suffers from strong computational overloads. In general, attention mechanism determines a mask to select the most representative features, discarding the remaining ones. In particular, the channel attention mechanism pursues a reduction in the number of channels of the feature maps produced by each cell. In this way, storage and computation costs are reduced.

Usually, attention-masked feature extraction is performed by a mechanism that provides its own weights $\boldsymbol{\omega}_{\text{att}}$ to produce the masked output of some input feature maps $\mathbf{X}$. Pooling layers, such as average (L^{avg}) and maximum ($L^{\max}$), fit perfectly with the attention mechanism, given their ability to extract salient features from the input data, and their combination is highly beneficial. Then, a spectral-based module, denoted as γ, aggregates the pooled data to generate the corresponding channel attention weights $\boldsymbol{\omega}_{\text{att}}$, as shown by the following equation:

$$\boldsymbol{\omega}_{\text{att}} = \sigma\left(\gamma\left(L^{\text{avg}}(\mathbf{X})\right) + \gamma\left(L^{\max}(\mathbf{X})\right)\right). \tag{10}$$

Function σ "binarizes" the weights to 1 and 0 and applies them to the original features, masking those with weight 0; thus, the masked feature data are $\boldsymbol{X}_{\text{att}} = \mathbf{X} \times \boldsymbol{\omega}_{\text{att}}$.

Complementary techniques, such as partial channel connections, aim to reduce the running memory utilization while limiting the overuse of unweighted operations. Similarly to attention, for each edge (i, j), the partial channel connection selects a number of top channels from the feature maps in n_i through a channel mask $M_{(i,j)} \in 0, 1^C$ (C denotes the total number of channels), following a proportion of $1/\kappa$. Consequently, only the channels within the range are evaluated in the operation space $\mathcal{O}$.

C. Challenges for HSI Classification

In this article, we specifically deal with the automatic generation of 1-D and 3-D CNNs for HSI classification. The generation of these models has been previously addressed by Chen et al. [81] in the context of HSI classification using a gradient descent-based NAS procedure. However, these algorithms suffer from several problems and shortcomings. In this regard, the computations are needed to evaluate all potential operations, and training a large number of parameters requires a trade-off between efficiency and complexity. This fact is aggravated when processing HSI scenes. Traditionally, this trade-off is achieved by regularization techniques, such as cutout or dropout. For instance, cutout suppresses randomly selected continuous band regions that negatively influence the generalization of the model, i.e., overfitting. Also, dropout [90] handles the overfitting problem by reinforcing the independence between neuron connectivities. This is accomplished by randomly deactivating neuronal activation during training, thus relaxing the amount of computation. However, the performance obtained by convolution layers is not as favorable, as there is a spatial relationship between the neurons that is not considered [45]. This introduces noise in the feature maps during the training procedure. Furthermore,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

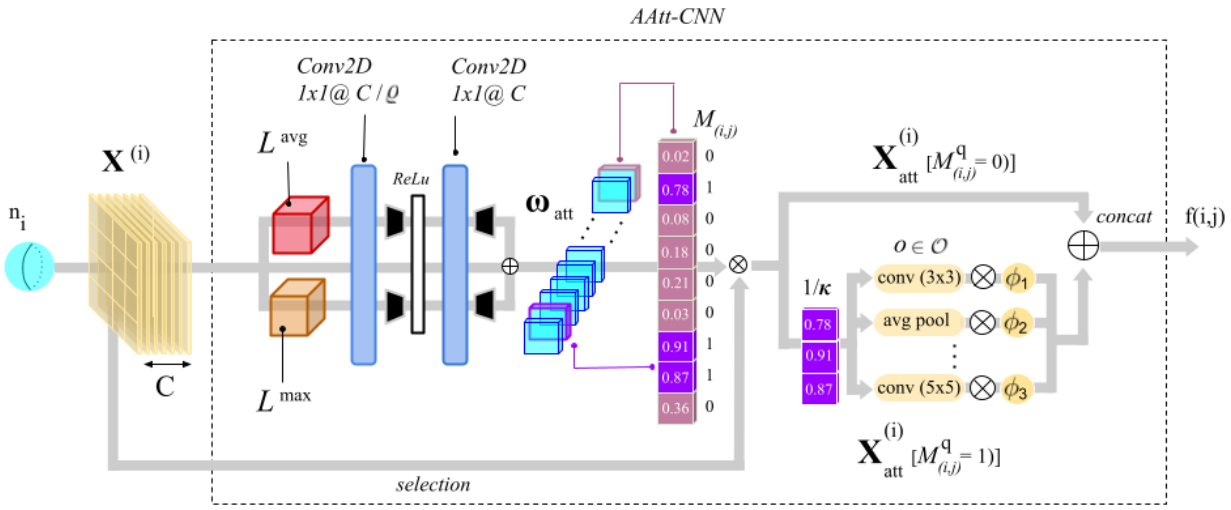


Fig. 4. Proposed attention mechanism (AAtt-CNN) for NAS. AAtt-CNN receives the feature maps of the previous layer $\mathbf{X}^{(l-1)}$ as input, with spectral depth dependent on the number of filters $F^{(l-1)}$. Subsequently, the pooling layers (L^{avg} and $L^{\max}$) are applied, among with a compression ratio. Then, the attention weights $\boldsymbol{\omega}_{\text{att}}^{(l)}$ are obtained. Finally, the most promising operations are selected taking into account the most representative features.

a new problem arises: how to avoid the use of generic models that do not fit the processing of HSIs. Recently, attention mechanisms have enhanced the performance of models used for classification tasks [26], [51]. In this regard, Xue and Qin [88] select the most promising operations through an attention mechanism. This improves feature extraction from the original data, reducing the complexity of search operations and stabilizing the architecture weights.

III. Proposed Methodology: AAtt-CNN

Addressing the posed challenges for HSI classification, this article presents a new efficient methodology based on NAS with a channel attention mechanism. The proposed method, known as AAtt-CNN, handles the processing and extraction of features over the spectral–spatial dimensions of the original HSI using a kernel of variable size (depending on the performed operation). The proposed method is built to automatically generate 1-D and 3-D architectures, reducing the required computations when extracting relevant features. Similar to classical NAS methods, a set of candidate operations that compose $\mathcal{O}$ are provided manually as the input, which is consistent with HSI processing basics. Different combinations of these operations form the architectures to be evaluated. They key point is the implementation of an attention mechanism to relax the computational cost of the architecture search, avoiding the selection of unfair operations and improving the model accuracy.

In this context, the proposed method works as follows. First, for each edge (i, j) in the cth cell, a pooling layer L^{avg} is implemented to average the coordinates of the C feature maps $\mathbf{X}^{(i)}$ in node n_i, into the output vector $\mathbf{x}^{\text{avg}}$

$$\mathbf{x}^{\text{avg}} = \circledast_C\left(\text{ReLU}\circledast_{C/\varrho}\left(L^{\text{avg}}\left(\mathbf{X}^{(i)}\right)\right)\right) \tag{11}$$

where $\circledast_{C/\varrho}$ and $\circledast_C$ are 2-D convolutions with kernels $1\times 1@C/\varrho$ and $1\times 1@C$, respectively. Next, a max-pooling layer $L^{\max}$ is implemented to to obtain the corresponding output vector $\mathbf{x}^{\max}$

$$\mathbf{x}^{\max} = \circledast_C\left(\text{ReLU}\circledast_{C/\varrho}\left(L^{\max}\left(\mathbf{X}^{(i)}\right)\right)\right). \tag{12}$$

Note that a compression is conducted following the compression ratio (ϱ). Furthermore, convolutions $\circledast_{C/\varrho}$ and $\circledast_C$ and rectified lineal unit (ReLU) function are shared, and the attention mechanism is performed by the combination of both pooling layers as shown below

$$\boldsymbol{\omega}_{\text{att}} = \mathbf{x}^{\text{avg}} + \mathbf{x}^{\max}. \tag{13}$$

Once the resulting masked features $\boldsymbol{X}_{\text{att}}^{(i)} = \mathbf{X}^{(i)} \times \boldsymbol{\omega}_{\text{att}}$ are obtained, they are processed along the channel dimension, where the $1/\kappa$ channels with higher attention weight are selected for the computation of the $|\mathcal{O}|$ operations, and the rest are directly copied to the output. This selection is supported by the mask $M_{(i,j)} \in \{0, 1\}^C$. The mask acts by selecting the $1/\kappa$ channels with higher attention weight value, assigning a 1 to its position, i.e., $M_{(i,j)}^q = 1$, where $q \in \kappa$. Once $1/\kappa$ features are retrieved, the space of operations $\mathcal{O}$ is further examined. Indeed, the architecture parameters $\phi_{i,j}$ are used to obtain the most suitable operation using (9). Finally, the channels that are not taken into account (i.e., those with $M_{(i,j)}^q = 0$) are skipped in the current node and added to compute the input function in the next node, j. In this sense, the overall calculation is given by the following equation:

$$\text{AAtt-CNN} = \mathbf{X}_{\text{att}}^{(i)}\left[M_{(i,j)}^q = 0\right] + \sum_{o\in\mathcal{O}} \mathbf{X}_{\text{att}}^{(i)}\left[M_{(i,j)}^q = 1\right]\phi_{ij}^o o_{ij}. \tag{14}$$

The overall procedure is illustrated in Fig. 4 for the computation of the output of a given node n_i. The aforementioned methodology is described in Algorithm 1 in a step-by-step fashion.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Proposed Methodology AAtt-CNN

Input: $\mathbf{X}^{(i)}$ features; C input channels

procedure AATT-CNN(x)
  $L^{avg}(\mathbf{X}^{(i)})$ (average pooling)
  $L^{max}(\mathbf{X}^{(i)})$ (maximum pooling)
  $\mathbf{x}^{avg} = \circledast_C(\text{ReLU} \circledast_{C/\varrho} (L^{avg}(\mathbf{X}^{(i)})))$
  $\mathbf{x}^{max} = \circledast_C(\text{ReLU} \circledast_{C/\varrho} (L^{max}(\mathbf{X}^{(i)})))$
  $\boldsymbol{\omega}_{att} = \mathbf{x}^{avg} + \mathbf{x}^{max}$
  $\boldsymbol{X}^{(i)}_{att} = \mathbf{X}^{(i)} * \boldsymbol{\omega}_{att}$
  Select $1/\kappa$ channels
  $M_{(i,j)} = Zeros(C)$
  for $q = 0$ to $len(\omega_{att})$ **do**
    if $\omega^{(q)}_{att} \in Top(1/\kappa)$ in $\boldsymbol{\omega}_{att}$ **then**
      $M^{q}_{(i,j)} = 1$
    end if
  end for
  Generate output space from $\mathcal{O}$
  Calculate AAtt-CNN using Eq. (14)
end procedure

IV. Experimental Results

A. Hyperspectral Datasets

The performance of the proposed AAtt-CNN has been evaluated on classification benchmarks widely used in the literature, in particular, the following hold: 1) Indian Pines (IP); 2) Kennedy Space Center (KSC); 3) University of Pavia (UP); 4) Salinas Valley (SV); and 5) University of Houston (UH). It is worth mentioning that the considered scenes have been captured using airborne sensors with different characteristics. Consequently, their spectral–spatial properties are significantly diverse on purpose to assess the robustness and generalization capability of the proposal. The ground-truth images are shown in Figs. 5 and 6 for IP, KSC, UP, SV, and UH, respectively.

1) Fig. 5(a) depicts the ground-truth information of **IP**, which was captured by the Airborne Visible Infra-Red Imaging Spectrometer (AVIRIS) sensor in the northwest of IN, USA, recording 20 m of land cover in each of its 145 × 145 pixels, and with a sensitivity in the spectral range of 400–2500 nm. The scene provides 224 bands, which are usually reduced to 200 by removing water absorption and noisy bands. The ground-truth comprises 16 classes of different types of crops and vegetation.
2) Fig. 5(b) depicts the ground-truth information of **KSC**, which was acquired over an area comprising wetland vegetation at the KSC, FL, USA, through the AVIRIS instrument. The image size is 512 × 614 pixels with 176 spectral bands after the suppression of noisy bands. Moreover, 13 classes are available that represent different land cover types. The wavelength range is 400–2500 nm, and the spatial resolution is 18 m.
3) Fig. 5(c) depicts the ground-truth information of **UP**, which was obtained by the Reflective Optics System Imaging Spectrometer (ROSIS-3) in Pavia, Italy. The dataset is composed of 610 × 340 pixels and 103 spectral bands. The image is captured in the range 420–860 nm with a resolution of 1.3 m per pixel. Concretely, nine urban classes represent the pixel materials, from asphalt to trees or metal sheets.
4) Fig. 5(d) depicts the ground-truth information of **SV**, which was gathered using the AVIRIS instrument over SV, CA, USA. The dataset comprises 512 × 217 pixels, with 204 bands and a spatial resolution of 3.7 m in the 400–2500-nm wavelength range. Similar to IP, 20 water absorption bands are removed. The ground truth comprises 16 classes, from vegetation to vineyard fields.
5) Fig. 6 depicts the ground-truth information of **UH**, which was retrieved in the urban area of the UH by the compact airborne spectrographic image (CASI). The data size is 349 × 1905 with 144 spectral bands, recorded in the wavelength range of 380–1050 nm, with a spatial resolution of 2.5 m. In this collection, ten of the 15 classes have more than 1000 samples, while the smallest class (the sixth class) has around 300 samples. In addition, there is a large amount of background pixels, which makes this dataset especially complex for classification.

Furthermore, different metrics have been considered for the evaluation of results, namely, the following hold: 1) class accuracy; 2) overall accuracy (OA); 3) average accuracy (AA); and 4) kappa coefficient (Kx100).

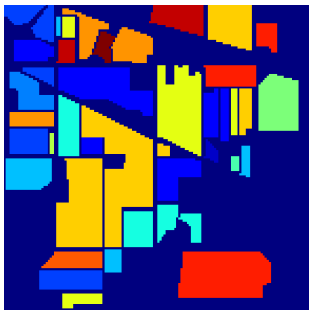
B. Experimental Settings

Two different approaches are evaluated, i.e., spectral models and spectral–spatial methods. The former provides valuable information on the classification performance of AAtt-CNN using only the spectral information contained in the pixel. Although this fails at describing the structure and spatial relationships of the observed materials and objects, it avoids one of the main criticisms against deep models, i.e., the use of spatial information overlapping training and test samples, resulting in unfair (overoptimistic) predictions. The use of spatial information, on the other hand, smooths the uncertainty of the spectral information caused by high variability and poor spatial resolution. Thus, the performance evaluating is extended to support classification with spectral–spatial information.

Two experiments are, therefore, conducted. The first compares popular spectral-based methods to our 1-D model, AAtt-CNN1D, specifically: random forest (**RF**), multiple linear regression (**MLR**), **SVM**, multilayer perceptron (**MLP**), **RNN**, long short-term memory (**LSTM**), gated recurrent unit (**GRU**), **CNN1D** [91], deep convolutional neural network 1D (**DCNN1D**) [92], and **AutoCNN1D** [81]. This experiment proves that the proposed AAtt-CNN is well suited to 1-D convolution-based architectures, which are particularly appealing for HSI processing because of their conceptual simplicity and ease of implementation.

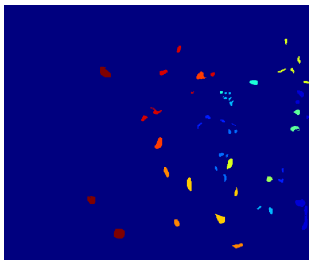
The second experiment compares SOTA spectral–spatial methods to our 3-D model, AAtt-CNN3D. To draw a fair and complete comparison, extended morphological profiles (EMPs) [93] for feature extraction followed by MLR for classification (**EMP-MLR**), EMP for feature extraction followed by SVM for classification (**EMP-SVM**), **pResNet** [94], spectral–spatial feature tokenization transformer (**SSFT**) [95],

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.



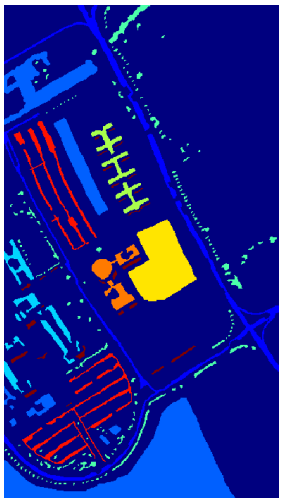
Color	Land cover type	Samples
	Background	10776
	Alfalfa	46
	Corn notill	1428
	Corn min	830
	Corn	237
	Grass/Pasture	483
	Grass/Trees	730
	Grass/pasture-mowed	28
	Hay windrowed	478
	Oats	20
	Soybeans notill	972
	Soybeans min	2455
	Soybean clean	593
	Wheat	205
	Woods	1265
	Bldg Grass Tree Drives	386
	Stone steel towers	93
	Total samples	21025

(a)



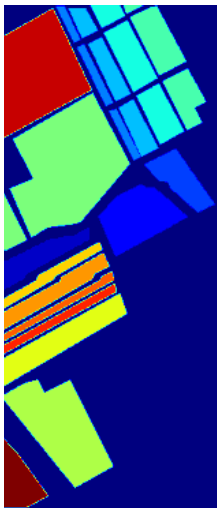
Color	Land cover type	Samples
	Background	309157
	Scrub	761
	Willow swamp	243
	CP hammock	256
	Slash pine	252
	Oak/Broadleaf	161
	Hardwood	229
	Swap	105
	Graminoid marsh	431
	Spartina marsh	520
	Cattail marsh	404
	Salt marsh	419
	Mud flats	503
	Water	927
	Total samples	314368

(b)



Color	Land cover type	Samples
	Background	164624
	Asphalt	6631
	Meadows	18649
	Gravel	2099
	Trees	3064
	Painted metal sheets	1345
	Bare Soil	5029
	Bitumen	1330
	Self Blocking Bricks	3682
	Shadows	947
	Total samples	207400

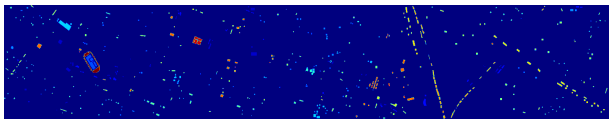
(c)



Color	Land cover type	Samples
	Background	56975
	Brocoli-green-weeds-1	2009
	Brocoli-green-weeds-2	3726
	Fallow	1976
	Fallow-rough-plow	1394
	Fallow-smooth	2678
	Stubble	3959
	Celery	3579
	Grapes-untrained	11271
	Soil-vinyard-develop	6203
	Corn-senesced-green-weeds	3278
	Lettuce-romaine-4wk	1068
	Lettuce-romaine-5wk	1927
	Lettuce-romaine-6wk	916
	Lettuce-romaine-7wk	1070
	Vinyard-untrained	7268
	Vinyard-vertical-trellis	1807
	Total samples	111104

(d)

Fig. 5. Ground truth of (a) IP, (b) KSC, (c) UP, and (d) SV scenes.



Color	Class	Samples	Color	Class	Samples
	Background	649816		Grass-healthy	1251
	Grass-stressed	1254		Grass-synthetic	697
	Tree	1244		Soil	1242
	Water	325		Residential	1268
	Commercial	1244		Road	1252
	Highway	1227		Railway	1235
	Parking-lot1	1233		Parking-lot2	469
	Tennis-court	428		Running-track	660
	Total samples				664845

Fig. 6. Ground truth of UH image.

convolution transformer mixer (**CTMixer**) [96], spectral–spatial residual network (**SSRN**) [97], robust self-ensembling network (**RSEN**) [98], **3-D CNN**, and **3-D AutoCNN** with cutout [81] have been considered.

Each experiment includes the architecture generated by the proposed AAtt-CNN method in the kth cell (c_k) for each dataset (see Figs. 7 and 8). We emphasize that our method comprises two different types of cells: *normal* and *reduction* cells. Normal cells apply padding $= 1$ and stride $= 1$ to keep the dimensions of feature maps. Meanwhile, reduction cells apply padding $= 2$ and stride $= 2$ to decrease the dimensions of feature maps. For comparative purposes, the number of cells is set to 3 and 4 for both experiments, with one and two reduction cells for architecture search and evaluation, respectively. Each cell contains four nodes, and it is connected to its two immediately preceding cells (c_{k-1} and c_{k-2}). The operation space $\mathcal{O}$ is composed of max and average pooling with 1×3 kernel, convolutions with 1×3, 1×5, 1×7, and 1×9 kernels, skip connections, zero, and identity operations for the 1-D approach. For the 3-D approach, $\mathcal{O}$ comprises 3×3 and 5×5 separated and dilated convolutions, 3×3 average and max pooling, skip connections, and identity operation.

The data are partitioned into different subsets, i.e., *training*, *validation*, and *test*. For each scene, a total of 200 samples have been selected to form the *training set*. It should be noted that samples are drawn randomly from the entire ground truth. For instance, Table I presents a selection of training samples from the evaluated datasets during an experiment, which is subject to variation with each iteration of the algorithm, resulting in a unique set of training samples for each launch. In this way, the selection of samples per class is avoided,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

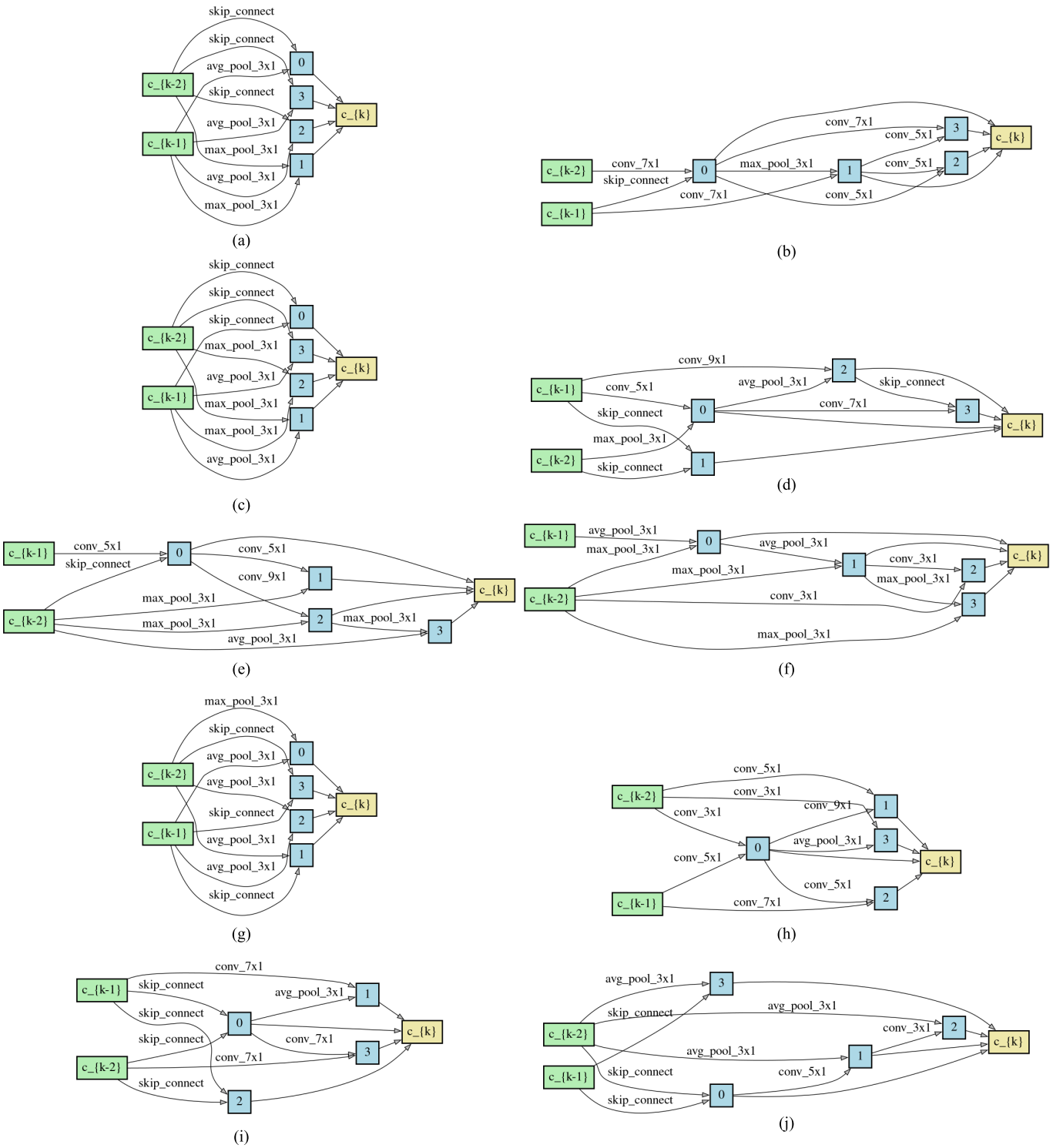


Fig. 7. Samples of architecture search-space operations obtained using the proposed 1-D approach for the studied datasets. Previous cells ($k - i$) are represented in green color, while the actual cell k is represented in yellow. Blue squares correspond to the nodes. (a) IP (normal). (b) IP (reduction). (c) UP (normal). (d) UP (reduction). (e) KSC (normal). (f) KSC (reduction). (g) SV (normal). (h) SV (reduction). (i) UH (normal). (j) UH (reduction).

with the objective of confronting the proposed model with a limited number of training samples, which may lead to overfitting problems. As a consequence, this involves the 1.95% of labeled data from IP, the 0.47% of UP, 3.84% of KSC, 0.37% of SV, and 1.33% of UH. Furthermore, to train `AAtt-CNN3D`, a total of 400 samples from UH have been randomly selected, involving the 2.66% of labeled data from UH. It must be highlighted that, compared with other current DL models [23], the proposed method considerably reduces the amount of information used during training. To form the *validation set*, a total of 100 labeled samples have been randomly selected from the entire ground truth provided by

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

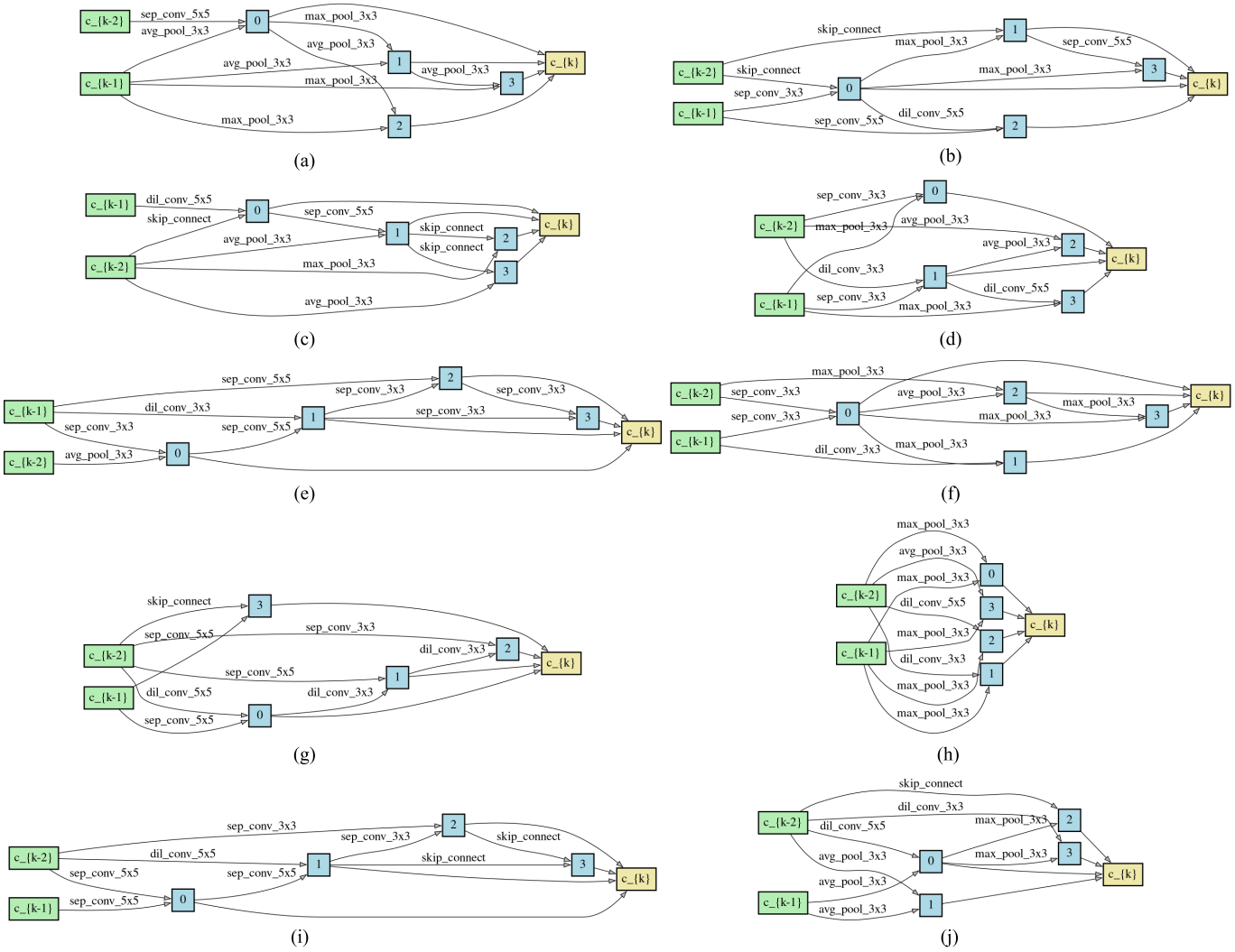


Fig. 8. Samples of architecture search-space operations obtained using the proposed 3-D approach for the studied datasets. Previous cells $(k-i)$ are represented in green color, while the actual cell k is represented in yellow. Blue squares correspond to the nodes. (a) IP (normal). (b) IP (reduction). (c) UP (normal). (d) UP (reduction). (e) KSC (normal). (f) KSC (reduction). (g) SV (normal). (h) SV (reduction). (i) UH (normal). (j) UH (reduction).

TABLE I

NUMBER OF TRAINING n_{TRAIN}, VALIDATION n_{VAL}, AND TEST n_{TEST} SAMPLES PER CLASS HSI OVER STUDIED DATASETS FOR THE 1-D APPROACHES

	IP			UP			KSC			SV			UH		
Class	n_{train}	n_{val}	n_{test}	n_{train}	n_{val}	n_{test}	n_{train}	n_{val}	n_{test}	n_{train}	n_{val}	n_{test}	n_{train}	n_{val}	n_{test}
0	1	0	45	35	18	6578	28	21	712	5	9	1995	16	11	1224
1	31	13	1383	88	40	18520	8	6	229	14	6	3706	16	9	1229
2	18	7	805	4	5	2090	12	4	240	6	3	1967	9	3	685
3	4	0	233	15	10	3039	9	2	241	4	0	1390	13	8	1223
4	6	3	474	5	5	1335	5	3	153	12	6	2660	12	7	1223
5	13	9	708	21	9	4999	2	6	221	15	7	3937	4	0	321
6	1	2	26	11	1	1318	3	3	99	12	10	3557	19	5	1244
7	12	4	462	20	10	3652	17	5	409	43	15	11213	20	9	1215
8	1	1	19	1	2	945	24	7	489	22	10	6171	22	11	1219
9	17	10	945				20	6	378	11	6	3261	14	8	1205
10	51	24	2378				16	9	394	2	1	1065	15	4	1216
11	5	6	582				25	12	466	11	6	1910	13	9	1211
12	5	3	197				31	16	880	2	1	913	9	6	454
13	29	11	1225							5	3	1062	5	4	419
14	5	4	377							29	15	7224	13	6	641
15	1	3	90							7	2	1798			
Total	200	100	9949	200	100	42476	200	100	4911	200	100	53829	200	100	14729

the HSI scenes (200 for the validation set of AAtt-CNN3D with UH). The remaining samples have been used for test.

Regarding the optimization process, two stages are differentiated, i.e., the *architecture search* and the *model training*. Regarding the former, $\boldsymbol{\omega}$ and $\boldsymbol{\phi}$ are adjusted to obtain the best network, using Adam optimizer [99] with 300 epochs and a batch size of 200 for AAtt-CNN1D and AAtt-CNN3D. Focusing on learning rates, AAtt-CNN1D uses $\xi = 4e-3$ with weight decay $3e-4$ and $\varepsilon = 3e-4$ with weight decay $3e-4$, while AAtt-CNN3D uses $\xi = 2.5e-2$ with weight decay $3e-4$ and $\varepsilon = 3e-4$ with weight decay $3e-4$. Once the best architecture is obtained, the model is retrained again to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CLASSIFICATION RESULTS OBTAINED BY 1-D METHODS FOR THE IP SCENE

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	1-D CNN	1-D DCNN	1-D AutoCNN	AAtt-CNN1D
0	4.89±6.94	6.67±5.96	17.78±16.69	0.0±0.0	1.78±2.59	21.33±28.13	3.11±4.35	13.78±12.91	**21.69±27.63**	19.58±19.42	19.11±19.04
1	51.82±6.61	58.79±4.17	**63.89±3.46**	36.94±9.52	38.29±4.58	55.13±6.53	51.47±12.69	55.3±3.05	54.14±9.77	60.16±8.24	61.03±8.91
2	26.72±11.84	41.49±14.06	47.59±13.33	0.02±0.05	10.21±12.91	11.94±8.25	15.91±13.8	43.09±4.04	**50.26±9.33**	44.12±12.07	49.34±5.37
3	15.88±4.74	14.82±4.0	29.83±8.67	0.17±0.35	10.62±8.03	24.3±12.23	26.83±19.61	3.46±2.01	30.89±11.69	25.35±15.82	**32.06±15.29**
4	46.82±25.25	71.31±7.12	77.15±7.55	4.72±9.45	40.98±12.23	28.11±28.46	48.33±10.38	55.49±10.68	46.16±21.31	**77.8±4.54**	61.51±24.01
5	89.32±8.14	91.55±1.47	93.2±3.35	81.59±18.34	86.98±6.35	**94.06±4.5**	88.9±7.48	86.07±3.28	82.95±10.75	90.99±4.86	92.46±2.59
6	6.84±9.19	24.19±29.79	31.77±39.32	0.0±0.0	21.05±26.6	19.09±29.79	33.28±41.16	0.0±0.0	24.84±26.35	35.63±36.89	**46.52±39.46**
7	92.17±4.54	80.69±18.54	88.29±14.05	95.51±5.46	80.42±13.35	88.28±9.86	92.84±12.85	89.42±4.71	94.33±6.75	**95.87±5.24**	95.48±4.74
8	5.56±11.11	13.22±21.56	15.38±23.67	0.0±0.0	7.66±10.85	3.16±6.32	16.43±23.78	0.0±0.0	1.05±3.16	5.31±9.74	**17.19±21.92**
9	32.82±8.37	48.33±6.39	55.68±4.66	0.12±0.25	22.14±11.32	61.57±12.84	42.95±10.57	49.57±3.47	41.17±21.43	55.93±9.41	**70.62±8.24**
10	80.23±5.48	65.13±4.15	63.07±4.06	**92.14±1.86**	77.78±5.72	71.89±4.0	73.46±8.98	66.07±3.3	69.44±8.23	68.73±9.5	66.57±5.24
11	29.08±4.9	32.91±9.42	36.21±7.51	2.08±2.25	12.75±6.45	32.07±11.01	25.91±17.95	31.34±3.58	30.64±17.15	**36.96±7.91**	34.74±10.6
12	74.63±14.63	83.1±8.69	90.91±2.58	0.0±0.0	64.98±3.85	83.1±12.43	72.98±31.02	88.99±6.48	78.8±18.87	87.33±10.85	**96.19±2.33**
13	93.79±3.32	90.86±3.1	89.39±3.29	**98.12±1.45**	87.26±4.79	92.25±5.21	91.72±2.07	96.81±4.51	85.59±7.73	84.9±4.19	92.73±3.64
14	20.37±17.84	27.13±19.31	25.18±19.32	9.14±14.7	19.3±17.22	11.73±7.55	21.42±11.8	17.41±4.23	30.2±15.28	**39.02±17.05**	32.84±4.21
15	23.7±19.91	35.43±31.17	48.26±39.42	0.0±0.0	7.83±9.97	41.96±34.34	46.3±37.89	63.67±24.15	**64.6±31.64**	48.02±42.87	52.14±42.64
OA	60.71±1.07	61.94±1.41	64.73±1.46	50.28±0.93	52.69±1.89	60.28±0.74	59.32±2.16	60.3±1.41	61.14±3.92	65.35±2.94	**67.1±1.66**
AA	43.42±1.94	49.1±4.21	54.6±4.16	26.29±0.61	36.88±4.32	46.25±3.23	46.99±6.73	46.9±1.75	50.42±5.11	54.73±4.22	**57.53±3.97**
K(x100)	54.19±1.14	56.34±1.63	59.75±1.63	39.96±1.18	44.29±2.78	54.04±0.78	52.8±2.86	54.38±1.59	55.14±4.65	60.41±3.23	**62.45±1.82**

TABLE III
CLASSIFICATION RESULTS OBTAINED BY 1-D METHODS FOR THE UP SCENE

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	1-D CNN	1-D DCNN	1-D AutoCNN	AAtt-CNN1D
0	81.06±5.27	83.15±3.77	79.12±3.79	**88.33±2.65**	79.27±3.59	80.08±3.6	78.76±4.98	79.83±3.89	83.76±2.74	83.4±3.4	86.94±3.56
1	93.28±3.17	94.04±1.11	96.14±1.13	93.14±2.25	93.65±1.73	94.6±1.33	95.54±0.61	91.57±2.17	92.96±3.47	93.32±3.18	**96.55±0.31**
2	37.31±11.69	**62.04±17.21**	57.73±14.04	8.85±11.29	51.15±8.16	57.66±13.68	52.84±10.44	56.97±12.1	48.97±20.97	61.52±5.05	62.01±10.24
3	76.57±11.07	78.38±2.61	80.4±6.66	71.59±9.47	**82.29±4.15**	78.05±3.81	74.97±5.11	79.38±4.5	79.2±6.52	78.86±4.38	77.41±8.41
4	89.71±10.57	93.37±8.1	98.37±0.73	98.18±0.69	**99.3±0.21**	99.26±0.33	98.55±0.62	95.27±7.08	99.2±0.29	98.25±0.98	99.19±0.5
5	35.55±8.73	57.69±3.17	57.37±4.31	12.82±8.21	37.72±7.61	41.49±5.7	41.22±5.64	58.15±7.29	57.56±10.49	**73.34±5.42**	67.6±3.52
6	65.59±9.71	41.2±13.5	**77.85±6.43**	0.0±0.0	72.52±5.27	70.22±9.73	67.5±12.18	62.41±13.33	61.52±17.47	64.56±9.15	67.55±8.19
7	72.79±9.33	79.63±8.78	80.8±9.4	81.53±9.02	**83.08±4.53**	78.05±6.9	81.88±7.28	79.66±5.23	73.69±11.16	76.86±7.29	78.7±6.54
8	99.51±0.29	98.94±0.51	99.7±0.04	74.04±36.69	98.92±0.14	99.34±0.26	**99.83±0.08**	97.68±2.5	98.63±0.57	97.69±2.3	79.79±39.89
OA	78.05±0.58	82.59±0.76	84.19±1.08	73.11±0.95	80.67±0.73	81.18±0.92	81.13±1.06	81.47±1.47	81.78±1.56	84.63±1.8	**85.86±1.26**
AA	72.37±2.25	76.49±1.22	80.83±1.74	58.72±4.98	77.54±1.32	77.64±1.91	76.79±2.01	77.88±1.62	77.28±2.38	**80.87±1.64**	79.53±5.26
K(x100)	70.1±1.16	76.55±0.97	78.65±1.48	62.66±1.55	73.79±1.01	74.45±1.25	74.31±1.51	75.49±1.94	75.72±2.09	79.7±2.26	**80.92±1.76**

TABLE IV
CLASSIFICATION RESULTS OBTAINED BY 1-D METHODS FOR THE KSC SCENE

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	1-D CNN	1-D DCNN	1-D AutoCNN	AAtt-CNN1D
0	94.31±2.93	95.05±2.08	92.65±2.86	96.24±1.33	95.79±2.17	**97.23±2.16**	93.08±4.15	90.14±2.29	90.48±3.21	88.96±7.77	93.38±2.56
1	63.35±19.35	74.33±16.78	72.61±13.51	0.08±0.17	23.85±34.83	3.03±5.85	34.95±30.63	60.96±5.48	83.96±8.5	**89.38±3.42**	89.25±2.38
2	85.86±6.82	86.38±6.63	86.76±6.05	0.0±0.0	6.23±12.46	0.0±0.0	18.85±37.7	74.73±7.71	72.87±21.6	83.74±6.75	**90.23±2.31**
3	56.03±9.91	52.9±11.39	54.54±14.69	0.0±0.0	2.32±4.65	0.0±0.0	1.56±1.75	52.54±6.48	41.01±20.92	60.08±10.93	**66.64±7.86**
4	43.04±7.76	58.52±8.31	61.19±5.23	0.0±0.0	6.24±12.48	0.0±0.0	9.26±13.11	**73.1±1.74**	37.84±14.33	55.35±9.88	34.86±20.4
5	29.51±16.67	34.74±14.39	37.15±12.5	0.0±0.0	0.0±0.0	0.0±0.0	3.18±6.36	34.86±7.75	33.64±12.4	**56.79±9.95**	48.81±19.24
6	69.65±20.59	85.73±11.75	84.92±14.48	0.0±0.0	5.15±10.3	0.0±0.0	14.23±28.45	**95.94±4.78**	53.11±18.88	68.58±19.96	43.04±28.38
7	61.41±8.52	88.78±4.43	**91.54±3.78**	19.23±22.92	22.89±22.72	12.37±20.58	37.13±23.01	85.5±3.66	73.6±6.53	88.4±4.58	89.7±5.35
8	85.34±3.55	91.15±3.01	93.19±3.31	66.21±19.64	67.96±12.5	33.24±38.23	84.17±3.91	91.08±2.07	86.27±8.84	94.3±4.69	**94.58±6.0**
9	71.34±11.25	88.25±3.7	89.03±3.52	9.93±17.44	62.36±28.95	19.43±24.62	47.58±26.74	94.74±1.74	92.22±3.05	95.28±2.37	**95.32±3.05**
10	91.69±2.62	93.43±3.74	92.73±2.67	88.36±2.42	88.4±6.7	76.21±11.63	86.2±2.77	95.4±2.2	93.8±3.84	**95.4±4.37**	93.56±5.21
11	73.14±8.98	76.39±2.83	81.72±1.92	1.69±1.12	19.48±28.66	2.89±1.66	31.92±33.03	82.3±1.52	80.84±6.7	**94.17±2.25**	91.3±3.28
12	99.66±0.12	98.44±0.82	98.49±0.79	98.55±0.81	99.19±0.7	98.8±0.66	98.85±0.59	99.86±0.01	99.92±0.05	100.0±0.0	**100.0±0.0**
OA	78.89±1.14	84.82±1.23	85.62±1.58	47.71±2.27	55.9±9.51	44.08±5.95	59.55±8.52	84.8±0.97	81.33±2.12	**88.4±2.02**	87.86±1.08
AA	71.1±2.58	78.77±2.36	79.73±2.99	29.25±1.9	37.99±11.46	26.4±5.27	42.58±10.62	79.32±1.48	72.27±3.71	**82.34±2.63**	79.28±3.14
K(x100)	76.44±1.31	83.07±1.38	83.98±1.77	39.31±2.84	49.2±11.23	34.47±7.9	53.64±10.33	83.08±1.08	79.16±2.38	**87.08±2.23**	86.47±1.2

adjust the final parameters, using also Adam optimizer with 500 and 150 epochs for `AAtt-CNN1D` and `AAtt-CNN3D`, respectively, and a batch size of 32. Focusing on the learning rate, `AAtt-CNN1D` uses $\xi = 4e-3$ with weight decay $3e-4$, while `AAtt-CNN3D` uses $\xi = 1.6e-3$ with weight decay $3e-4$. These values have been inspired by Chen et al. [81] and Liu et al. [89].

Finally, the compression ratio conducted by the attention mechanism has been deeply studied to produce the best result. In this sense, different values have been tested (2, 4, 8, 16, and 32). Fig. 9 depicts the obtained results in terms of OA over the IP scene, where $\varrho = 8$ provides the best OA (%). This study elucidates the process of determining the value of ϱ that best represents the attention weights ω_{att}, which is a challenging task to accomplish through manual engineering, owing to its inherent operational complexity.

C. Experiments and Discussion

1) `AAtt-CNN1D` Evaluation: This experiment draws a comparison (using the considered metrics) to determine which 1-D classifier conducts a better classification. Obtained results are provided in Tables II–VI. It should be noted that, in general, for all experiments conducted over the HSI scenes, the method is stable and performs well in minority classes despite random sampling.

Table II provides the classification results obtained for the IP scene. It is noteworthy that IP is extremely challenging, due to the difficulty of classifying crop areas, which have been

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS OBTAINED BY 1-D METHODS FOR THE SV SCENE

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	1-D CNN	1-D DCNN	1-D AutoCNN	AAtt-CNN1D
0	96.45±3.95	97.5±0.58	96.7±0.77	77.23±38.77	93.2±1.93	96.91±1.13	95.91±2.05	96.67±1.85	**97.95±1.36**	96.75±1.56	97.34±2.91
1	98.28±1.46	98.4±1.02	97.26±1.58	93.27±7.64	98.69±1.19	94.36±4.04	96.3±3.87	98.22±1.18	98.41±1.21	**99.26±0.45**	99.22±0.34
2	76.3±13.12	**89.03±10.47**	83.02±12.82	0.0±0.0	79.02±9.93	77.65±7.96	79.4±11.64	85.25±3.3	86.12±13.77	79.46±21.58	87.03±12.82
3	92.26±2.94	99.06±0.74	98.57±1.0	28.18±34.01	99.14±0.34	97.95±1.28	97.64±1.19	**99.65±0.25**	96.18±8.76	99.09±0.56	98.82±0.79
4	92.93±6.42	96.27±1.59	94.44±4.49	43.91±32.79	93.87±3.91	93.21±2.37	96.33±0.97	95.32±0.51	91.17±5.28	**97.21±1.46**	96.48±1.66
5	97.76±2.34	98.38±2.22	99.08±0.97	**99.8±0.0**	98.42±2.14	98.89±1.08	99.5±0.21	97.48±1.1	97.52±1.58	99.68±0.17	99.25±0.44
6	98.47±0.8	99.23±0.31	97.8±1.97	94.03±10.02	97.76±1.16	96.85±2.07	98.73±0.75	97.81±0.9	99.01±0.32	**99.35±0.25**	99.0±0.37
7	73.49±8.62	76.26±6.36	77.1±7.01	**90.6±9.2**	70.77±5.3	84.33±2.7	77.19±15.06	77.09±2.92	75.95±11.48	75.82±10.02	74.06±4.94
8	98.99±0.15	98.53±0.56	98.31±0.93	99.03±0.21	97.89±1.06	97.19±1.67	96.51±2.87	97.69±0.46	97.77±1.11	99.05±0.75	**99.63±0.28**
9	78.56±3.37	84.63±3.66	82.95±3.84	58.04±21.65	82.25±3.47	79.54±6.97	80.03±4.28	84.69±2.44	78.37±14.45	**87.54±3.78**	82.44±4.54
10	59.26±22.98	85.77±6.59	77.42±18.66	0.04±0.08	54.24±11.4	44.24±36.85	49.04±40.5	**89.69±1.24**	71.92±23.32	89.15±2.3	83.9±7.49
11	94.61±3.41	98.9±0.82	**99.34±0.58**	15.61±31.21	97.48±2.06	95.99±6.41	97.77±1.23	97.12±1.48	88.34±17.31	96.99±4.2	97.81±2.72
12	98.29±0.37	**99.52±0.33**	98.95±0.5	28.54±37.41	98.14±0.64	98.29±1.1	97.65±3.19	98.5±0.36	80.92±28.25	98.36±1.08	97.83±3.26
13	61.62±32.12	68.4±34.36	66.54±33.44	6.42±12.84	71.68±35.94	70.91±35.53	72.05±36.03	88.37±1.69	88.61±6.92	**90.61±5.26**	83.31±15.28
14	47.64±9.3	61.89±7.3	65.64±7.67	13.37±18.38	57.31±7.26	34.12±10.69	44.03±24.68	49.89±5.6	54.38±19.32	63.47±10.73	**68.45±4.64**
15	78.96±7.5	83.08±7.17	83.66±6.07	14.01±23.93	75.81±10.28	68.96±6.85	81.69±7.48	86.01±5.2	79.78±16.25	89.26±5.84	**93.97±4.69**
OA	81.62±1.42	86.34±1.26	86.24±1.72	62.85±2.19	82.77±1.5	81.45±1.93	82.32±1.95	84.93±0.73	83.63±2.41	87.15±1.37	**87.34±0.49**
AA	83.99±2.69	89.68±2.85	88.55±2.73	47.63±4.13	85.35±2.78	83.09±4.29	84.99±4.04	89.97±0.58	86.4±4.36	**91.32±1.63**	91.16±0.98
K(x100)	79.46±1.57	84.79±1.41	84.67±1.9	57.5±2.5	80.8±1.67	79.21±2.22	80.24±2.18	83.2±0.82	81.74±2.74	85.7±1.5	**85.92±0.54**

TABLE VI
CLASSIFICATION RESULTS OBTAINED BY 1-D METHODS FOR THE UH SCENE

Class	RF	MLR	SVM	MLP	RNN	LSTM	GRU	1-D CNN	1-D AutoCNN	AAtt-CNN1D
0	82.6±5.67	90.93±4.71	**95.17**±2.39	65.25±32.6	89.39±4.93	89.77±6.24	89.32±6.73	82.28±7.82	93.78±5.08	94.84±5.04
1	93.45±5.98	93.62±4.98	91.85±4.5	86.15±0.97	94.82±5.57	95.23±4.36	96.2±5.06	**96.28**±4.67	92.75±5.07	92.73±5.04
2	92.41±2.05	**99.8**±0.15	98.25±1.42	32.9±40.5	98.61±1.07	98.22±0.9	99.42±0.21	96.28±3.54	99.53±0.3	99.77±0.15
3	92.35±1.64	94.52±2.74	93.04±2.19	90.77±8.36	**96.45**±2.27	94.12±2.56	94.98±2.9	92.38±2.66	93.29±3.28	92.05±1.4
4	89.36±8.14	97.47±1.13	94.51±2.6	**98.53±1.23**	97.24±0.79	94.74±4.52	94.51±4.54	96.07±1.81	96.64±2.31	96.61±1.68
5	73.21±17.48	77.76±6.63	85.58±4.73	58.89±32.26	86.15±5.96	85.41±4.54	77.81±16.82	62.42±25.32	**88.68±5.05**	88.61±5.92
6	71.7±10.19	**86.2±2.63**	76.09±7.96	71.33±11.96	77.17±4.19	72.32±5.71	81.35±3.01	55.86±26.22	70.34±4.31	75.19±7.14
7	56.82±7.09	54.68±4.64	**69.11±8.91**	46.91±14.54	64.03±3.83	55.51±8.26	53.61±9.76	65.47±4.51	57.26±11.84	63.44±11.56
8	63.18±11.14	71.45±7.26	**77.02±4.27**	59.11±35.89	69.79±5.2	70.63±4.03	65.61±4.26	54.93±23.72	74.01±5.51	73.98±9.5
9	57.83±11.26	77.36±7.32	70.33±9.0	10.06±15.29	73.49±14.2	70.24±16.27	67.4±13.71	80.75±16.31	79.68±3.22	**84.01±2.89**
10	67.7±7.1	76.17±4.86	75.82±5.31	42.64±28.74	73.56±6.43	63.38±6.89	60.48±7.77	56.7±31.6	**80.76±7.1**	77.97±7.79
11	56.49±7.0	57.32±5.34	65.11±7.62	22.48±15.85	65.94±5.08	64.06±7.48	52.48±12.34	19.28±18.24	74.3±3.19	**75.77±5.87**
12	7.68±4.39	**34.91±11.19**	31.49±13.74	1.53±3.05	19.9±10.11	15.39±9.36	10.26±6.48	12.16±17.9	25.65±8.42	24.88±4.32
13	77.73±15.94	86.12±17.87	81.86±15.11	59.38±32.84	**90.55±12.71**	77.9±14.24	85.17±14.61	83.18±18.06	91.03±14.66	88.93±16.45
14	88.62±7.19	97.7±0.35	**99.02±0.18**	78.77±39.39	98.86±0.41	98.4±0.85	97.88±0.39	97.36±1.51	98.89±1.06	98.43±0.78
OA	72.84±1.52	80.43±0.53	81.03±1.95	57.23±2.62	80.42±1.52	77.23±1.11	76.01±0.82	70.84±6.32	81.59±0.76	**82.64±0.85**
AA	71.41±2.42	79.73±0.95	80.28±1.6	54.98±2.25	79.73±1.73	76.35±1.73	75.1±1.73	70.09±7.19	81.11±1.18	**81.81±1.08**
K(x100)	70.6±1.64	78.82±0.57	79.49±2.09	53.63±2.81	78.82±1.64	75.37±1.22	74.03±0.89	68.41±6.87	80.08±0.82	**81.22±0.92**

TABLE VII
CLASSIFICATION RESULTS OBTAINED BY 3-D METHODS FOR THE IP SCENE

Class	EMP-MLR	EMP-SVM	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
0	49.27±40.91	49.31±42.43	41.33±15.31	21.49±18.05	17.99±17.63	27.27±34.5	12.46±22.28	35.98±32.52	33.14±40.99	**90.71±12.37**
1	77.75±4.46	80.7±3.3	56.36±1.56	64.26±9.94	87.51±8.83	71.35±8.09	79.05±10.59	82.49±6.84	**88.28±3.21**	87.06±3.87
2	73.94±6.68	81.53±10.8	57.5±2.27	75.44±11.42	**86.81±8.6**	62.91±6.66	73.92±16.97	64.88±14.94	79.86±14.67	66.98±16.6
3	45.2±22.21	50.23±26.25	**70.61±5.36**	53.25±23.2	66.89±26.23	35.48±9.31	38.96±21.69	45.67±20.51	56.57±29.64	69.68±40.17
4	81.98±2.97	**82.92±2.23**	80.17±3.27	72.08±18.1	73.98±23.57	53.82±25.07	75.46±16.0	58.91±16.72	71.13±16.66	74.27±5.87
5	94.04±5.72	90.27±7.4	84.46±3.2	96.87±1.7	98.37±0.46	**98.83±0.34**	94.84±5.2	90.86±8.48	95.58±4.27	94.64±3.59
6	56.27±46.02	57.75±47.21	**73.33±13.13**	60.0±48.99	59.26±48.4	49.49±40.43	33.7±43.84	48.26±42.02	48.18±44.87	67.14±41.8
7	98.38±3.24	94.93±9.83	96.21±1.38	97.6±2.86	**100.0±0.0**	100.0±0.0	99.53±0.69	94.54±7.28	99.77±0.46	97.67±4.22
8	34.74±43.35	38.95±47.73	0.0±0.0	9.47±16.44	25.26±31.12	22.69±27.8	8.48±13.13	24.29±31.89	20.0±40.0	**58.0±47.5**
9	77.87±4.91	84.49±2.36	64.77±5.27	82.21±5.13	85.83±7.11	75.04±8.51	75.52±8.84	86.89±4.65	91.74±3.8	**93.87±3.83**
10	86.84±4.3	87.19±2.52	82.96±4.71	86.58±6.38	91.67±2.32	**95.45±1.31**	82.76±6.9	92.72±1.57	93.7±2.22	94.21±2.0
11	58.49±20.84	57.72±20.63	40.24±4.36	45.53±21.3	71.56±24.68	58.57±15.42	68.04±19.14	68.58±16.91	73.7±21.19	**78.4±13.18**
12	99.11±1.06	98.31±1.49	64.43±20.11	95.41±5.29	**99.2±1.03**	98.81±1.89	95.41±8.34	92.99±13.36	94.75±4.65	93.26±7.42
13	97.28±1.7	95.25±6.09	98.09±0.67	92.48±5.5	**99.48±0.42**	97.73±4.22	96.38±3.15	98.89±1.28	98.2±0.96	94.7±3.2
14	58.37±12.56	63.81±10.98	63.68±4.43	74.45±20.18	72.54±18.84	65.41±14.76	60.64±19.6	**75.44±13.8**	50.99±34.23	45.66±43.52
15	68.87±34.97	65.41±32.87	73.26±14.71	72.74±38.79	79.12±39.6	12.15±13.05	50.4±42.3	57.62±26.55	70.57±29.54	**91.25±7.71**
OA	81.8±1.85	83.14±1.96	73.6±1.61	79.33±3.43	88.12±3.33	80.35±2.44	80.66±2.91	85.29±1.45	89.01±2.02	**90.08±0.67**
AA	72.4±3.13	73.67±2.53	65.46±3.24	68.74±4.78	75.97±1.33	64.06±3.48	65.35±6.85	69.94±4.83	72.88±8.22	**81.09±2.7**
K(x100)	79.25±2.1	80.78±2.24	69.73±1.75	76.31±4.0	86.42±3.82	77.23±2.85	77.83±3.37	82.7±1.75	87.16±2.37	**88.19±0.79**

imaged in early stages of growth, and are, therefore, affected by strong mixing phenomena. Nevertheless, the proposed AAtt-CNN1D substantially improves the rest of the methods with superior OA, AA, and Kx100 values. In this scenario, the proposed approach gains significant accuracy improvement for classes with a low number of samples, such as six Grass/pasture-mowed and eight Oats. Also, this improvement is maintained for classes with a higher number of samples, e.g., class three Corn, nine Soybeans notill, and 12 Wheat. This has a beneficial effect on the final metrics obtained by our AAtt-CNN1D, proving its effectiveness for this classification benchmark. Also, methods, such as SVM and 1-D AutoCNN, obtain scores that are close to those obtained by the proposed method.

Table III provides the results for the UP dataset, which is characterized by its rich spatial information. The obtained

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
CLASSIFICATION RESULTS OBTAINED BY 3-D METHODS FOR THE UP SCENE

Class	EMP-MLR	EMP-SVM	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
0	95.41±4.57	95.86±2.25	87.86±2.05	95.11±2.6	**95.52±0.69**	95.11±2.27	95.08±2.46	92.11±2.4	94.14±2.53	95.34±2.0
1	98.16±0.53	98.13±0.9	96.63±0.86	98.54±1.21	99.25±0.74	**99.81±0.14**	96.2±4.2	98.88±0.72	92.78±1.89	99.15±0.79
2	85.12±5.61	87.83±5.86	36.6±7.69	68.2±23.0	67.43±26.56	44.58±27.41	69.92±18.36	59.99±13.85	80.6±2.23	**90.34±4.27**
3	93.33±4.19	94.88±1.75	90.99±1.5	87.31±6.15	91.55±6.06	**98.34±0.3**	90.84±6.93	93.35±4.35	83.42±1.12	75.02±9.21
4	99.25±0.46	**99.63±0.11**	98.49±1.51	97.77±2.51	99.45±0.47	80.11±6.8	99.25±2.01	97.1±4.3	99.13±0.11	98.29±1.72
5	73.16±6.15	80.61±4.62	72.42±5.95	95.17±3.61	97.15±3.0	77.7±1.2	89.34±10.88	93.97±3.24	95.62±0.05	**97.74±1.94**
6	96.23±2.41	**96.43±1.56**	70.39±2.94	47.03±39.32	84.41±12.07	46.98±17.45	82.36±14.01	54.23±12.48	87.31±1.25	94.37±3.55
7	94.52±4.07	93.91±4.79	85.03±2.24	89.75±4.98	92.15±1.87	96.78±2.21	79.64±18.68	88.77±5.64	**98.39±4.52**	98.1±0.65
8	92.67±3.45	96.84±6.26	95.16±4.19	77.0±35.88	92.49±6.06	77.4±38.74	**98.34±2.11**	82.93±21.6	63.02±3.36	69.84±13.88
OA	93.35±1.12	94.58±0.83	87.29±1.31	92.45±2.77	**95.09±1.27**	90.64±1.21	91.65±2.64	91.79±0.98	93.88±1.86	95.04±1.53
AA	91.98±0.84	**93.79±1.51**	81.51±1.91	83.99±9.57	91.05±3.17	79.65±3.06	89.0±3.27	84.59±2.97	88.19±2.72	90.91±2.69
K(x100)	91.11±1.51	92.77±1.13	82.99±1.79	89.92±3.76	93.48±1.71	87.4±1.64	89.16±3.37	89.58±1.26	92.15±2.29	**93.71±1.94**

TABLE IX
CLASSIFICATION RESULTS OBTAINED BY 3-D METHODS FOR THE KSC SCENE

Class	EMP-MLR	EMP-SVM	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
0	95.59±3.45	98.27±1.62	97.44±1.15	54.18±44.48	**100.0±0.0**	95.18±1.01	97.72±2.16	99.72±0.29	99.24±0.71	96.77±4.9
1	60.55±14.14	73.37±14.3	57.06±6.85	17.49±15.83	**97.11±3.21**	37.75±4.61	90.35±9.95	82.18±9.58	91.2±6.91	96.83±4.19
2	61.82±23.1	81.1±12.39	69.67±7.65	0.84±1.03	98.29±1.67	87.15±2.57	95.88±2.52	92.8±7.6	94.19±5.53	**98.67±1.8**
3	43.12±13.13	66.66±4.98	33.06±5.24	5.4±2.74	75.59±11.38	37.45±18.29	91.68±5.97	78.38±7.19	**92.38±4.67**	86.28±13.69
4	31.46±25.77	46.49±21.84	55.51±5.37	4.86±3.32	74.43±16.0	43.8±8.77	81.88±9.97	91.57±7.51	**99.76±0.43**	88.6±9.84
5	49.07±21.68	61.34±11.69	72.18±2.91	0.0±0.0	87.71±9.41	22.38±12.13	91.55±8.02	88.02±8.54	95.57±6.12	**100.0±0.0**
6	28.06±35.44	54.56±36.86	80.4±3.26	0.0±0.0	57.53±33.06	54.04±24.71	89.91±10.76	96.64±7.55	100.0±0.0	**100.0±0.0**
7	55.8±14.5	79.8±7.65	91.65±4.22	3.53±4.03	98.4±2.01	55.71±1.4	96.96±4.31	95.81±5.42	96.13±3.04	**98.53±2.71**
8	84.45±2.09	90.01±4.21	85.47±1.32	39.84±48.79	99.67±0.55	91.2±2.56	98.16±3.77	98.46±3.19	100.0±0.0	**100.0±0.0**
9	45.01±7.13	64.18±6.31	40.15±10.2	26.59±11.81	100.0±0.0	56.95±2.73	99.35±1.19	100.0±0.0	**100.0±0.0**	99.83±0.34
10	93.07±3.45	93.99±3.71	94.64±2.66	56.02±44.98	99.39±0.99	82.41±2.43	98.28±2.83	100.0±0.0	100.0±0.0	**100.0±0.0**
11	73.06±4.23	86.27±4.21	76.37±12.06	28.07±11.58	98.87±1.62	79.6±17.02	**99.37±1.63**	99.33±1.07	96.43±5.32	98.9±2.2
12	97.06±1.04	98.08±0.45	99.27±0.66	74.29±30.98	100.0±0.0	100.0±0.0	100.0±0.0	100.0±0.0	100.0±0.0	**100.0±0.0**
OA	73.83±1.95	84.06±1.35	80.19±1.62	35.78±5.95	96.05±0.56	75.31±1.27	96.77±0.83	95.98±0.85	97.61±0.39	**98.16±1.45**
AA	62.93±4.38	76.47±4.01	73.3±1.14	23.93±6.11	91.31±2.3	64.89±2.23	94.70±1.39	94.07±1.39	**97.29±0.58**	97.26±2.07
K(x100)	70.76±2.19	82.24±1.5	77.88±1.81	26.64±7.32	95.6±0.62	72.32±1.44	96.41±0.93	95.53±0.95	97.33±0.44	**97.96±1.61**

TABLE X
CLASSIFICATION RESULTS OBTAINED BY 3-D METHODS FOR THE SV SCENE

Class	EMP-MLR	EMP-SVM	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
0	99.14±0.12	99.13±0.07	89.22±0.61	98.61±2.76	97.88±0.72	82.45±15.91	**99.59±0.72**	81.76±6.68	94.15±6.02	85.91±20.1
1	99.69±0.09	99.05±0.63	99.45±0.51	99.8±0.2	**100.0±0.0**	99.43±0.39	99.54±1.35	94.88±6.07	98.36±2.09	96.72±4.84
2	90.89±9.4	92.41±10.11	76.91±2.68	86.57±17.14	93.77±10.18	**97.02±3.78**	82.03±20.05	91.35±4.62	93.59±7.15	97.01±4.28
3	96.99±4.74	98.93±0.55	97.22±2.92	95.27±4.55	98.96±0.75	**99.28±0.24**	98.61±1.57	88.31±7.49	98.56±2.3	92.6±7.1
4	94.11±4.3	93.65±4.29	98.79±0.66	88.37±12.46	94.5±2.56	**99.39±0.11**	97.13±3.98	95.6±4.4	98.61±1.15	98.82±1.13
5	99.49±0.28	99.32±0.36	98.8±0.27	99.0±2.01	99.96±0.07	**100.0±0.0**	99.81±0.49	99.76±0.63	99.67±0.16	99.54±0.38
6	97.75±1.26	96.87±1.24	96.2±0.97	95.23±5.78	99.09±0.52	99.19±0.6	**99.62±0.58**	97.54±2.03	97.96±2.39	99.38±0.81
7	83.32±4.24	85.12±4.77	83.47±2.71	90.72±5.76	92.35±3.83	86.92±4.14	81.13±8.86	88.48±3.13	90.98±1.13	**96.96±2.09**
8	99.46±0.58	99.42±0.55	97.29±1.62	**99.86±0.15**	99.79±0.17	99.74±0.32	98.96±2.3	98.76±1.7	99.74±0.16	99.85±0.28
9	87.81±9.73	85.11±6.9	81.01±4.03	89.2±5.03	91.25±4.37	84.65±10.8	91.58±3.72	95.2±5.57	**99.78±0.31**	97.94±3.39
10	94.7±2.11	94.94±2.21	83.99±5.57	94.82±4.1	**95.12±3.23**	88.65±8.75	83.38±28.97	77.33±8.43	79.51±4.37	75.63±37.96
11	98.13±3.75	95.5±9.0	92.57±2.38	93.74±4.07	96.81±1.13	**100.0±0.0**	98.72±3.02	89.66±9.04	99.84±0.22	93.35±7.1
12	**98.46±0.5**	98.27±0.72	97.11±0.37	96.25±5.04	97.67±0.86	77.11±38.61	95.3±5.09	87.92±9.43	90.86±0.98	78.63±22.92
13	89.07±2.67	90.18±3.02	94.5±1.78	58.99±32.21	91.46±4.51	96.72±5.44	**96.95±3.15**	90.53±8.71	87.17±3.14	92.64±6.64
14	83.75±3.3	85.44±5.2	82.11±2.36	63.98±24.78	83.87±7.71	86.78±7.66	79.72±7.47	81.78±11.24	**97.4±2.02**	96.84±1.42
15	88.09±13.69	90.69±10.84	**98.34±1.72**	95.72±2.18	93.39±7.39	87.14±2.59	95.37±2.84	61.12±14.38	77.63±4.31	75.25±38.25
OA	91.79±1.32	92.21±1.01	90.15±0.71	89.58±3.53	94.38±0.86	92.53±1.32	91.16±1.64	90.68±1.28	94.65±0.7	**96.16±1.77**
AA	93.8±0.7	94.0±0.87	91.68±0.48	90.38±3.88	**95.37±0.76**	92.78±2.66	93.59±2.35	88.18±1.25	93.99±2.01	92.32±3.58
K(x100)	90.87±1.47	91.33±1.12	89.04±0.78	88.36±3.99	93.74±0.96	91.68±1.48	90.16±1.82	89.56±1.45	93.87±0.81	**95.69±1.99**

results show a very similar behavior with respect to the results obtained for the IP scene, where improvements in accuracy are observed in a considerable fraction of the nine classes of the data set. Mainly, the classes zero Asphalt, one Meadows, and six Bitumen shown the improvements of +3.54, +3.23, and +2.99 compared with the second most efficient method, 1-D AutoCNN. The proposed `AAtt-CNN1D` improves the results obtained by 1-D AutoCNN for the majority of classes, with some exception, such as five Bare Soil and eight Shadows. Even though the AA metric is better for the 1-D AutoCNN, the results obtained by the proposed `AAtt-CNN1D` are the most interesting.

The results obtained for KSC are shown in Table IV. This scene is challenging due to the scarcity of training samples, as it can be seen in Fig. 5(b), where the classes are identified in a small portion of the image, and most of the samples remain as unlabeled. This inherent difficulty complicates the discrimination between classes, which are highly similar in terms of spectral signatures. In this regard, vegetation classes, such as one Willow swamp, five Oak/Broadleaf, ten Salt marsh,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
CLASSIFICATION RESULTS OBTAINED BY 3-D METHODS FOR THE UH SCENE

Class	EMP-MLR	EMP-SVM	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
0	94.15±1.43	95.5±1.48	94.69±1.49	88.16±7.72	96.99±2.42	96.95±1.5	**98.08±1.57**	96.94±1.77	93.1±4.9	97.46±1.45
1	93.83±5.24	93.93±5.1	96.57±0.4	87.69±2.9	95.06±4.17	**98.85±0.94**	90.36±4.38	97.04±0.61	98.01±2.59	98.71±1.19
2	95.39±2.13	98.13±1.43	99.74±0.35	97.45±2.52	99.22±0.62	99.91±0.12	**100.0±0.0**	99.97±0.06	99.53±0.57	99.36±0.66
3	93.17±2.1	96.05±2.51	98.51±0.51	92.8±1.5	96.15±2.18	96.93±1.22	95.54±1.39	**97.99±1.84**	95.29±0.82	96.64±1.36
4	97.13±1.93	98.74±0.89	99.42±0.23	92.53±6.9	99.95±0.1	100.0±0.0	97.67±0.88	97.72±0.6	98.81±0.75	**100.0±0.0**
5	90.35±8.1	91.56±8.42	77.76±5.13	49.67±23.05	90.62±10.11	90.9±12.35	77.36±8.39	83.68±1.56	84.92±6.72	**94.77±3.78**
6	91.9±2.47	90.85±3.42	88.81±2.47	87.41±3.34	94.77±1.85	94.4±1.76	91.4±2.22	88.9±3.04	94.36±1.86	**97.04±1.27**
7	54.1±9.44	76.46±7.88	85.96±0.67	59.17±12.1	83.99±6.6	**91.72±2.01**	84.22±1.68	88.38±4.5	80.37±6.55	88.3±2.9
8	79.94±3.75	84.79±3.6	87.48±1.09	83.14±9.86	91.43±4.13	**90.72±1.83**	87.93±2.58	86.75±2.16	85.26±1.97	90.35±3.1
9	90.13±2.08	90.5±2.63	89.84±0.95	62.68±13.08	94.04±4.88	**99.42±0.8**	94.92±1.78	90.05±1.69	93.54±1.29	98.53±1.42
10	77.09±6.97	87.56±1.33	89.16±2.29	80.18±11.86	94.19±1.65	92.13±3.17	82.78±4.21	76.9±4.65	91.4±4.66	**97.6±0.61**
11	59.0±8.83	76.09±9.65	87.47±4.88	73.87±17.32	90.0±3.85	92.67±0.24	88.31±4.52	86.49±5.43	92.62±1.95	**95.46±0.66**
12	66.44±4.08	60.44±7.12	86.96±2.05	89.09±5.37	92.47±2.7	49.53±19.04	87.77±2.92	92.41±2.39	90.71±4.5	**96.54±1.8**
13	90.66±8.28	89.74±10.25	94.05±2.76	89.79±12.11	99.33±1.22	96.72±4.17	94.25±4.0	91.54±4.82	99.62±0.39	**99.76±0.37**
14	99.56±0.34	99.66±0.12	99.4±0.88	83.14±17.32	**100.0±0.0**	86.99±17.02	99.97±0.06	99.59±0.37	97.91±2.99	99.97±0.06
OA	84.2±1.36	89.11±1.27	92.12±0.57	81.52±2.43	94.25±1.36	94.28±0.64	91.62±0.72	91.47±1.07	92.86±1.18	**96.43±0.41**
AA	84.86±1.18	88.67±1.21	91.72±0.37	81.12±3.14	94.55±1.41	92.72±1.54	91.37±0.92	91.62±0.97	93.03±1.09	**96.7±0.44**
K(x100)	82.92±1.47	88.22±1.37	91.47±0.61	80.01±2.63	93.78±1.47	93.81±0.7	90.93±0.78	90.78±1.16	92.28±1.27	**96.14±0.45**

TABLE XII
NUMBER OF TRAINABLE PARAMETERS FOR REPRESENTATIVE 3-D METHODS. THE VALUES ARE REPRESENTED IN THOUSANDS K

Dataset	pResNet	SSFT	CTMixer	RSEN	SSRN	3-D CNN	3-D AutoCNN	AAtt-CNN3D
Indian Pines (IP)	1125.9	153.2	625.7	2674.7	191.4	364.2	103.5	**91.3**
University of Pavia (UP)	1110.2	152.8	621.2	2468.8	141.3	216.5	83.8	**66.3**
Kennedy Space Center (KSC)	1121.7	153.0	624.1	2622.4	178.9	327.2	**61.6**	64.9
Salinas Valley (SV)	1126.5	153.2	625.7	2682.9	193.5	370.3	76.7	**48.9**
University of Houston (UH)	1117.6	678.5	623.6	2559.0	278.1	978.6	**42.1**	76.1

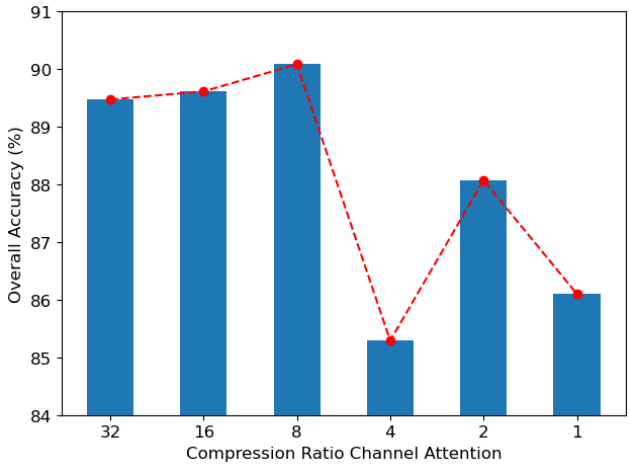


Fig. 9. Different values for the compression ratio (ϱ) have been tested to obtain the best result in terms of OA over the IP scene.

and 11-Mud Flats, are most effectively recognized using 1-D AutoCNN. On the contrary, the proposed AAtt-CNN1D manages to substantially improve the classes corresponding to marshes, such as seven Graminoid marsh, eight Spartina marsh, or nine Cattail marsh. Also, notable improvements are observable in two CP hammock and three Slash pine. In turn, methods, such as MLP, RNN, LSTM, or GRU, are unable to obtain good classifications due to the complexity of the image, obtaining about half the accuracy over the total (100%).

The results for SV are shown in Table V. The challenge of this image lies in the different stages of lettuce crop maturation, which affects the spectral variability of the samples. In this case, the results are more consistent for all methods. Even so, the proposed AAtt-CNN1D achieves very significant improvements for the classes eight Soil-vinyard-develop, and 14 and 15 Vinyards. This leads our method to outperform the remaining ones. Furthermore, the 1-D AutoCNN method achieves values that are close to those obtained by our proposed approach, reaching good accuracy with Fallow and Brocoli classes.

Finally, Table VI provides the obtained results over UH scene. This scene is characterized by having spatially disjoint test and training sets, which makes the samples very far apart and very different from each other; i.e., there is a large spectral–spatial variability of the samples. Nonetheless, once again, the proposed method is able to obtain the best classification results.

As a summary, all 1-D methods considered in this experiment extract the spectral features from the data correctly, with the proposed AAtt-CNN1D outperforming, in general, the classification accuracy of the spectral classifiers.

2) AAtt-CNN3D Evaluation: In this experiment, the proposed approach is evaluated to determine its performance with respect to other 3-D classifiers. Obtained results can be observed in Tables VII–XI. Once more, for all experiments conducted over the HSI scenes, the method is stable and performs well in minority classes despite random sampling.

The first dataset to be evaluated is IP. The results are shown in Table VII. As in the first experiment with AAtt-CNN1D, the main advantage of the proposed method is the improvement in the extraction of complex class features. For the zero Alfalfa and eight Oats, which are small classes composed of 46 and 20 samples and represent only 0.21% and 0.09% of the total image pixels, our Att-CNN3D obtains outstanding accuracy results, which have a beneficial impact on OA (90.08 ± 0.67), AA (81.09 ± 2.7), and Kx100 (88.19 ± 0.79).

Second, the results with the UP scene are presented in Table VIII. In this case, the classifiers can take into account

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

the rich spatial information provided by the scene to reduce the uncertainty. Consequently, most classifiers achieve high accuracy, such as EMP-SVM, CTMixer, and 3-D AutoCNN, with the OA values of 94.58 ± 0.83, 95.09 ± 1.27, and 93.88 ± 1.86, respectively. Nonetheless, our AAtt-CNN3D achieves pretty good values of 95.04 ± 1.53, 90.91 ± 2.69, and 93.71 ± 1.94 of OA, AA, and Kx100, respectively.

Focusing on KSC, unlike 1-D techniques, processing with 3-D kernels allows differentiation and extraction of complex spatial–spectral characteristics. Thus, it can be seen in Table IX that accuracies have been increased compared with those reported in Table IV. The proposed AAtt-CNN3D is the best classifier for the vast majority of the classes and obtains the values of 98.16 ± 1.45, 97.26 ± 2.07, and 97.96 ± 1.61 in terms of OA, AA, and Kx100, respectively.

The results obtained for the SV scene are reported in Table X. Following the same pattern of classification enhancement observed in previous scenes, our AAtt-CNN3D maintains this trend. Moreover, the results with the SV scene demonstrate an added robustness to the proposal. This is related to the absence of classes with very small representation in the image, opposite to IP. Thus, classes, which are dominant in the scene, such as seven Grapes-untrained, obtain an improvement of +4.61 with respect to CTMixer (the second best method). The final results of the proposed method are 96.16 ± 1.77 (OA), 92.32 ± 3.58 (AA), and 95.69 ± 1.99 (Kx100).

Finally, Table XI provides the obtained results for UH scene. Once more, the proposed AAtt-CNN3D reaches the best OA value, providing also the highest values for AA and Kappa coefficient. The deviations shows the stability of the model.

As a summary of the results with 3-D methods, their accuracy has risen with respect to the first experiment because of the spatial information provided by neighboring pixels. The generated architectures for the 3-D classifiers obtain high accuracy results and are successful for classification compared with recent methods, such as SSFT, CTMixer, and RSEN.

In addition, Table XII shows the number of trainable parameters for the 3-D classifiers. The proposed method achieves a significant parameter reduction. Specifically, our Att-CNN3D reduces the number of trainable parameters (in contrast with 3-D AutoCNN) by a factor of 11.78%, 20.88%, and 36.24% for IP, UP, and SV, respectively. Conversely, KSC requires a slightly superior (5.08%) number of parameters, and UH requires an increment of (80.76%) but produces higher accuracy.

D. Architectural Search Space

The process of searching cell structures (hence, operations between nodes) for $I = 4$ is depicted for both 1-D and 3-D approaches in Figs. 7 and 8, respectively. In addition, for each HSI scene, both types of cells are shown (i.e., normal and reduction cells). In the case of the AAtt-CNN1D, convolutions with small kernels and pooling layers are selected more frequently than weight-free operations. As a consequence, the performance of the architecture is enhanced, while solving gradient degradation problems that lead to downgraded performance. For example, this reduction in unweighted operations is repeated for all the datasets in Fig. 7. Thus, it is also noticeable that significantly fewer skip connections are used in the reduction cells than in the normal cells.

On the other side, for the AAtt-CNN3D in Fig. 8, the search space reveals the appearance of separated and dilated convolutions. This is due to the processing properties of 3-D classifiers, where convolutions allow the separation or expansion of the convolution kernels and consequently reduce the complexity of the training. Given the need to spatially reduce the input data, pooling layers with higher kernels are included in Fig. 8. Finally, the reduction of unweighted operations is even more noticeable for the 3-D (spectral–spatial) cases, practically dropping to a minimum in both cells.

V. Conclusion

In this article, we have presented a new HSI classification method based on a channel attention mechanism for 1-D and 3-D CNNs, called AAtt-CNN. Our newly proposed method automatically generates convolutional neural architectures for HSI processing. The most promising operations that build such architecture are selected from a given range of input operations, including those that are most optimal for the extraction of both spatial and spectral features. To address the computation and classification challenges related to the complexity of HSI data, our proposal aims to boost the performance in both sides through a channel attention mechanism. In addition, our method addresses an usual shortcoming related to the selection of operations that do not have associated weights. An experimental evaluation has been conducted using four well-known HSI datasets with different specifications. The results obtained by the proposed method are compared with those obtained by SOTA classification methods. Our comparison reveals that the proposed 1-D and 3-D approaches outperform the respective competitors. Furthermore, a major advantage is provided by the significant improvements observed in the classification of minority classes (neither neglecting nor compromising the rest of the classes). From an experimental point of view, we can conclude that our newly developed AAtt-CNN improves HSI classification in both imbalanced and balanced scenarios.

References

[1] C. M. Gevaert, J. Suomalainen, J. Tang, and L. Kooistra, "Generation of spectral–temporal response surfaces by combining multispectral satellite and hyperspectral UAV imagery for precision agriculture applications," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 3140–3146, Jun. 2015.

[2] P. Singh et al., "Hyperspectral remote sensing in precision agriculture: Present status, challenges, and future trends," in *Hyperspectral Remote Sensing*. Amsterdam, The Netherlands: Elsevier, 2020, pp. 121–146.

[3] X. Zhang, Y. Sun, K. Shang, L. Zhang, and S. Wang, "Crop classification based on feature band set construction and object-oriented approach using hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 9, no. 9, pp. 4117–4128, Sep. 2016.

[4] C. Weber et al., "Hyperspectral imagery for environmental urban planning," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 1628–1631.

[5] Y. Fang, Z. Hu, L. Xu, A. Wong, and D. A. Clausi, "Estimation of iron concentration in soil of a mining area from UVA-based hyperspectral imagery," in *Proc. 10th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, Sep. 2019, pp. 1–5.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

[6] J. Ma, W. Sun, G. Yang, and D. Zhang, "Hydrological analysis using satellite remote sensing big data and CREST model," *IEEE Access*, vol. 6, pp. 9006–9016, 2018.

[7] M. J. Khan, H. S. Khan, A. Yousaf, K. Khurshid, and A. Abbas, "Modern trends in hyperspectral image analysis: A review," *IEEE Access*, vol. 6, pp. 14118–14129, 2018.

[8] J. Xue, Y.-Q. Zhao, Y. Bu, W. Liao, J. C.-W. Chan, and W. Philips, "Spatial-spectral structured sparse low-rank representation for hyperspectral image super-resolution," *IEEE Trans. Image Process.*, vol. 30, pp. 3084–3097, 2021.

[9] W. Dong, C. Zhou, F. Wu, J. Wu, G. Shi, and X. Li, "Model-guided deep hyperspectral image super-resolution," *IEEE Trans. Image Process.*, vol. 30, pp. 5754–5768, 2021.

[10] A. Maffei, J. M. Haut, M. E. Paoletti, J. Plaza, L. Bruzzone, and A. Plaza, "A single model CNN for hyperspectral image denoising," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2516–2529, Apr. 2020.

[11] J. M. Bioucas-Dias, A. Plaza, G. Camps-Valls, P. Scheunders, N. Nasrabadi, and J. Chanussot, "Hyperspectral remote sensing data analysis and future challenges," *IEEE Geosci. Remote Sens. Mag.*, vol. 1, no. 2, pp. 6–36, Jun. 2013.

[12] Y. Tarabalka, J. A. Benediktsson, and J. Chanussot, "Spectral–spatial classification of hyperspectral imagery based on partitional clustering techniques," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 8, pp. 2973–2987, Aug. 2009.

[13] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. Chanussot, and J. C. Tilton, "Advances in spectral–spatial classification of hyperspectral images," *Proc. IEEE*, vol. 101, no. 3, pp. 652–675, Mar. 2013.

[14] K. Y. Ma and C.-I. Chang, "Iterative training sampling coupled with active learning for semisupervised spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8672–8692, Oct. 2021.

[15] J. M. Haut, M. E. Paoletti, J. Plaza, J. Li, and A. Plaza, "Active learning with convolutional neural networks for hyperspectral image classification using a new Bayesian approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6440–6461, Nov. 2018.

[16] C.-I. Chang and K. Y. Ma, "Band sampling of kernel constrained energy minimization using training samples for hyperspectral mixed pixel classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5510723.

[17] J. M. Haut, M. Paoletti, J. Plaza, and A. Plaza, "Cloud implementation of the K-means algorithm for hyperspectral image analysis," *J. Supercomput.*, vol. 73, no. 1, pp. 514–529, Jan. 2017.

[18] M. E. Paoletti, J. M. Haut, X. Tao, J. P. Miguel, and A. Plaza, "A new GPU implementation of support vector machines for fast hyperspectral image classification," *Remote Sens.*, vol. 12, no. 8, p. 1257, Apr. 2020. [Online]. Available: https://www.mdpi.com/2072-4292/12/8/1257

[19] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4085–4098, Nov. 2010.

[20] H. Yin, X. Jiao, Y. Chai, and B. Fang, "Scene classification based on single-layer SAE and SVM," *Exp. Syst. Appl.*, vol. 42, no. 7, pp. 3368–3380, May 2015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957417414007854

[21] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Feb. 2019.

[22] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "Multilevel superpixel structured graph U-Nets for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5516115.

[23] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[24] L. Zhang, L. Zhang, and B. Du, "Deep learning for remote sensing data: A technical tutorial on the state of the art," *IEEE Geosci. Remote Sens. Mag.*, vol. 4, no. 2, pp. 22–40, Jun. 2016.

[25] M. E. Paoletti, J. M. Haut, N. S. Pereira, J. Plaza, and A. Plaza, "Ghostnet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 12, pp. 10378–10393, Dec. 2021.

[26] M. E. Paoletti, S. Moreno-Alvarez, and J. M. Haut, "Multiple attention-guided capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5520420.

[27] A. Maffei, J. M. Haut, M. E. Paoletti, J. Plaza, L. Bruzone, and A. Plaza, "Efficient convolutional neural network for spectral–spatial hyperspectral denoising," in *Proc. 10th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, 2019, pp. 1–4.

[28] J. L. Garcia, M. E. Paoletti, L. I. Jimenez, J. M. Haut, and A. Plaza, "Efficient semantic segmentation of hyperspectral images using adaptable rectangular convolution," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[29] S. K. Roy, R. Mondal, M. E. Paoletti, J. M. Haut, and A. Plaza, "Morphological convolutional neural networks for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 8689–8702, 2021.

[30] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.

[31] J. M. Haut, M. E. Paoletti, J. Plaza, and A. Plaza, "Fast dimensionality reduction and classification of hyperspectral images with extreme learning machines," *J. Real-Time Image Process.*, vol. 15, no. 3, pp. 439–462, Oct. 2018.

[32] P. Soille, *Morphological Image Analysis: Principles and Applications*, vol. 2. Berlin, Germany: Springer, 1999.

[33] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, Jan. 2017.

[34] S. Moreno-Alvarez, M. E. Paoletti, G. Cavallaro, J. A. Rico, and J. M. Haut, "Remote sensing image classification using CNNs with balanced gradient for distributed heterogeneous computing," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[35] H. Li, H. Huang, Z. Ye, and H. Li, "Hyperspectral image classification using adaptive weighted quaternion Zernike moments," *IEEE Trans. Signal Process.*, vol. 70, pp. 701–713, 2022.

[36] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[37] S. K. Roy, J. M. Haut, M. E. Paoletti, S. R. Dubey, and A. Plaza, "Generative adversarial minority oversampling for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5500615.

[38] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[39] S. Zhong, S. Chen, C.-I. Chang, and Y. Zhang, "Fusion of spectral–spatial classifiers for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5008–5027, Jun. 2021.

[40] V. Slavkovikj, S. Verstockt, W. De Neve, S. Van Hoecke, and R. Van de Walle, "Hyperspectral image classification with convolutional neural networks," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1159–1162.

[41] M. Lin, Q. Chen, and S. Yan, "Network in network," 2013, *arXiv:1312.4400*.

[42] X. Ying, "An overview of overfitting and its solutions," *J. Phys., Conf. Ser.*, vol. 1168, Feb. 2019, Art. no. 022022.

[43] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.

[44] A. Y. Ng, "Feature selection, L_1 vs. L_2 regularization, and rotational invariance," in *Proc. 21st Int. Conf. Mach. Learn.*, 2004, p. 78.

[45] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Neighboring region dropout for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 6, pp. 1032–1036, Jun. 2020.

[46] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Hyperspectral image classification using random occlusion data augmentation," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 11, pp. 1751–1755, Nov. 2019.

[47] H. Fu et al., "A novel band selection and spatial noise reduction method for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5535713.

[48] M. E. Paoletti, J. M. Haut, X. Tao, J. Plaza, and A. Plaza, "FLOP-reduction through memory allocations within CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5938–5952, Jul. 2021.

[49] M. E. Paoletti, X. Tao, J. M. Haut, S. Moreno-Álvarez, and A. Plaza, "Deep mixed precision for hyperspectral image classification," *J. Supercomput.*, vol. 77, no. 8, pp. 9190–9201, Aug. 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.

[50] L. Mou and X. X. Zhu, "Learning to pay attention on spectral domain: A spectral attention module-based convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 110–122, Jan. 2020.

[51] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Visual attention-driven hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 8065–8080, Oct. 2019.

[52] T. Alipour-Fard, M. E. Paoletti, J. M. Haut, H. Arefi, J. Plaza, and A. Plaza, "Multibranch selective kernel networks for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 6, pp. 1089–1093, Jun. 2021.

[53] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 1, pp. 1997–2017, 2019.

[54] P. Ren et al., "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–34, May 2022.

[55] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evol. Comput.*, vol. 10, no. 2, pp. 99–127, Jun. 2002.

[56] E. Real et al., "Large-scale evolution of image classifiers," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2902–2911.

[57] L. Xie and A. Yuille, "Genetic CNN," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 1379–1388.

[58] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Evolving deep convolutional neural networks for image classification," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 394–407, Apr. 2020.

[59] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing CNN architectures using the genetic algorithm for image classification," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3840–3854, Sep. 2020.

[60] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, no. 1, pp. 237–285, Jan. 1996.

[61] Y. Li, "Deep reinforcement learning: An overview," 2017, *arXiv:1701.07274*.

[62] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," 2016, *arXiv:1611.02167*.

[63] B. Zoph and Q. Le, "Neural architecture search with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.

[64] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.

[65] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1–9.

[66] K. Kandasamy, W. Neiswanger, J. Schneider, B. Poczos, and E. P. Xing, "Neural architecture search with Bayesian optimisation and optimal transport," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 1–10.

[67] H. Zhou, M. Yang, J. Wang, and W. Pan, "Bayesnas: A Bayesian approach for neural architecture search," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7603–7613.

[68] N. Nguyen and J. M. Chang, "CSNAS: Contrastive self-supervised learning neural architecture search via sequential model-based optimization," *IEEE Trans. Artif. Intell.*, vol. 3, no. 4, pp. 609–624, Aug. 2022.

[69] E. Lomurno, S. Samele, M. Matteucci, and D. Ardagna, "Pareto-optimal progressive neural architecture search," in *Proc. Genetic Evol. Comput. Conf. Companion*, Jul. 2021, pp. 1726–1734.

[70] C. Liu et al., "Progressive neural architecture search," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 19–34.

[71] L. Wang, Y. Zhao, Y. Jinnai, Y. Tian, and R. Fonseca, "Neural architecture search using deep neural networks and Monte Carlo tree search," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 6, pp. 9983–9991.

[72] B. Baker, O. Gupta, R. Raskar, and N. Naik, "Accelerating neural architecture search using performance prediction," 2017, *arXiv:1705.10823*.

[73] H. J. Escalante, "Automated machine learning—A brief review at the end of the early years," 2020, *arXiv:2008.08516*.

[74] A. Balaji and A. Allen, "Benchmarking automatic machine learning frameworks," 2018, *arXiv:1808.06492*.

[75] I. Drori et al., "Automatic machine learning by pipeline synthesis using model-based reinforcement learning and a grammar," 2019, *arXiv:1905.10345*.

[76] J.-M. Perez-Rua, M. Baccouche, and S. Pateux, "Efficient progressive neural architecture search," 2018, *arXiv:1808.00391*.

[77] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Aging evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, vol. 2, 2019, p. 2.

[78] T. Elsken, J. H. Metzen, and F. Hutter, "Efficient multi-objective neural architecture search via Lamarckian evolution," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–23. [Online]. Available: https://openreview.net/forum?id=ByME42AqK7

[79] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[80] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[81] Y. Chen, K. Zhu, L. Zhu, X. He, P. Ghamisi, and J. A. Benediktsson, "Automatic design of convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7048–7066, Sep. 2019.

[82] X. Dong and Y. Yang, "Searching for a robust neural architecture in four GPU hours," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 1761–1770.

[83] L. Li, M. Khodak, N. Balcan, and A. Talwalkar, "Geometry-aware gradient algorithms for neural architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–25.

[84] B. Wu et al., "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10734–10742.

[85] X. Chu, T. Zhou, B. Zhang, and J. Li, "Fair DARTS: Eliminating unfair advantages in differentiable architecture search," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 465–480.

[86] Y. Xu et al., "Partially-connected neural architecture search for reduced computational redundancy," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 9, pp. 2953–2970, Sep. 2021.

[87] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1294–1303.

[88] Y. Xue and J. Qin, "Partial connection based on channel attention for differentiable neural architecture search," *IEEE Trans. Ind. Informat.*, vol. 19, no. 5, pp. 6804–6813, May 2023.

[89] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," 2018, *arXiv:1806.09055*.

[90] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[91] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[92] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[93] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

[94] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Feb. 2019.

[95] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.

[96] J. Zhang, Z. Meng, F. Zhao, H. Liu, and Z. Chang, "Convolution transformer mixer for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[97] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[98] Y. Xu, B. Du, and L. Zhang, "Robust self-ensembling network for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Aug. 19, 2022, doi: 10.1109/TNNLS.2022.3198142.

[99] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:54:22 UTC from IEEE Xplore. Restrictions apply.